
ACTING UNDER UNCERTAINTY

INSTRUCTOR: MOAYAD ALNAMMI

malnammi.github.io

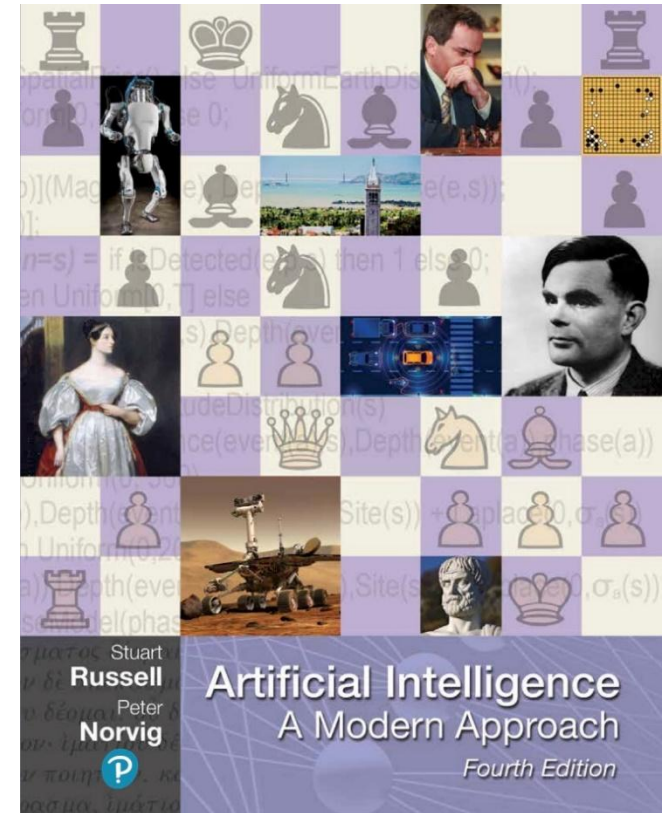
Notice

- Slides prepared by Moayad Alnammi – malnammi.github.io
- Some figures adapted from CS188: Introduction to Artificial Intelligence, UC Berkeley (Dan Klein, Pieter Abbeel) — ai.berkeley.edu
- Some figures are obtained from the [AIMA Book](#)
- Some content adapted from Dr. ElSayed El-Alfy
- Some content adapted from Dr. Richard Lathrop (UC Irvine)

This deck is published at malnammi.github.io under CC BY-NC-SA 4.0. Reuse and adaptation require attribution to the author and the sources above; commercial use is not permitted; adaptations must carry the same license.

Resources & Reading

- Textbook website: <http://aima.eecs.berkeley.edu/>
- Read Chapter 12 and 13 in AIMI 4/E



Outline

- Representing uncertainty
- Review of basic probability concepts
- Statistical modelling of a problem with many variables:
 - Full-joint distribution, exact inference, parameter estimation
 - Reducing complexity by exploiting assumptions: independence, conditional independence
 - Bayesian networks, exact inference, approximate inference, parameter estimation

Uncertainty Overview

Short history of AI

1940-1950: Early days	1943: McCulloch & Pitts: Neural Nets, Boolean circuit model of brain 1950: Turing's "Computing Machinery and Intelligence"
1950—70: Excitement I	1950s: Early AI programs: chess, checkers (RL), theorem proving 1956: Dartmouth summer meeting: "Artificial Intelligence" adopted 1965: Robinson's complete algorithm for logical reasoning General-problem solvers could not computationally scale to "difficult" problems.
1970—90: Knowledge-based approaches (expert systems)	Shift from "general" to domain-specific problem solvers. Leverage domain knowledge to reduce computational complexity. 1969—79: Early development of knowledge-based systems 1980—88: Expert systems industry booms. Million dollars in 1980 to billions in 1988. 1988—93: Expert systems industry busts: "AI Winter" [models did not account for uncertainty and did not learn from experience]
1986 — 2012: Statistical approaches	Resurgence of probability, focus on uncertainty (probability) General increase in technical depth Agents and learning systems "AI Spring"
2012— ____: Excitement II	Availability of big data and shift towards Machine Learning big compute, advances in GPU, deep learning. AI used in many industries

Propositional Logic vs Probability Theory

Language	Ontological Commitment (What exists in the world)	Epistemological Commitment (What an agent believes about facts)
Propositional logic	facts	true/false/unknown
First-order logic	facts, objects, relations	true/false/unknown
Temporal logic	facts, objects, relations, times	true/false/unknown
Probability theory	facts	degree of belief $\in [0, 1]$
Fuzzy logic	facts with degree of truth $\in [0, 1]$	known interval value

- Their ontological commitments are the same.
- Their epistemological commitments differ
 - **Logic agent** believes true, false, or no opinion.
 - **Probabilistic agent** has a numerical degree of belief between 0 (false) and 1 (true).

Intro

- Logic and theorem-proving still has uses in applications that need formal verification like: hardware verification of computer components, and formal software verification.
- However, for many real-world environment, there are too many unknowns.
- Using logic where statements are either true or false, does not capture the reality fully as we can varying degrees of belief about statements. The agent can update the degree of belief as he gathers more evidence.
- **Uncertainty** is very common in real applications, e.g. – decision systems (medical, repair, business, industrial, gardening, etc.).
- How to deal with it?
 - Inefficient and sometimes impractical approach is to attempt to model every possible scenario.
 - Use another methodology that naturally considers uncertainty, e.g. probabilistic modeling and inference – (**uncertain reasoning**)

Uncertainty in the world

Uncertainty due to

- Overwhelming complexity
- Lack of knowledge
- Partial observability
- Randomness (a bandage for all sources of unknowns in our problem)

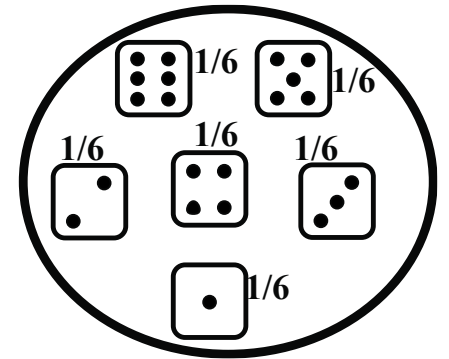
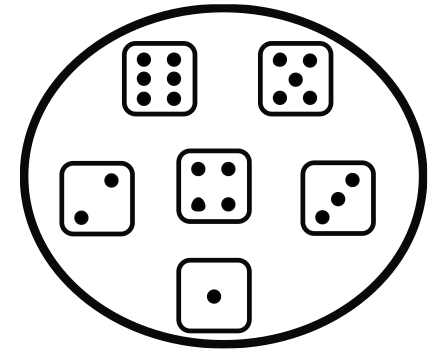
Probability gives

- Natural way to describe our assumptions about unknown/unobserved information
- Rules for how to combine information
- Indicates **degrees of belief** about the state of the environment
- Changes with more evidence:
 - $P(\text{EventA} \mid \text{EventB}) = 0.2$
 - $P(\text{EventA} \mid \text{EventB}, \text{EventC}, \text{EventD}) = 0.5$

Probability Basics

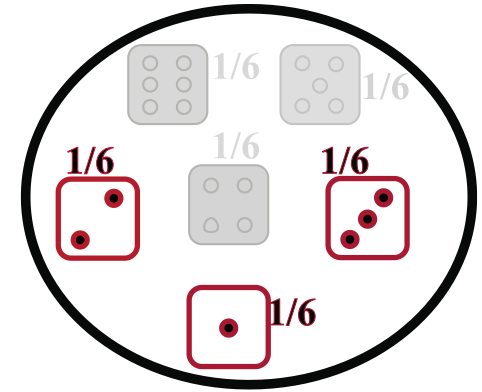
Probability Basics [1]

- Begin with a set Ω called the **sample space**
 - E.g., 6 possible rolls of a die, $\Omega = \{1, 2, 3, 4, 5, 6\}$
- A **sample/outcome** is an element $\omega \in \Omega$.
- A **probability model** assigns a number $P(\omega)$ to each outcome ω
 - E.g., $P(1) = P(2) = P(3) = P(4) = P(5) = P(6) = 1/6$.
- Probability model must satisfy
 - $0 \leq P(\omega) \leq 1$
 - $\sum_{\omega \in \Omega} P(\omega) = 1$



Probability Basics [2]

- An **event** A is any subset of Ω . $P(A) = \sum_{\omega \in A} P(\omega)$
 - e.g., $P(\text{die roll} < 4) = P(1) + P(2) + P(3) = 1/6 + 1/6 + 1/6 = 1/2$
- $P(\neg A) = 1 - P(A)$ -- **complement event**
- Special cases
 - $P(A) = 0$ iff $A = \emptyset$ -- **impossible/empty event**
 - $P(A) = 1$ iff $A = \Omega$ -- **certain event**
- $P(A \vee B) = P(A) + P(B) - P(A \wedge B)$
- If A and B are **disjoint** events (**mutually exclusive**; occurrence of one excludes the other, i.e. can not happen at the same time):
 - $P(A \wedge B) = 0$
 - $P(A \vee B) = P(A) + P(B)$



Random Variables

- **Random variables:** For ease of syntax, we will use variables to denote aspects of the world for which we would like to model uncertainty (probabilities).
- Variables can be assigned values that represent the outcome of an event.
- When we say that the variable X follows distribution D . We denote this as $X \sim D$
- Discrete Example: Coin flip
 - Variable R , the result of the coin flip
 - $R \in \{heads, tails\}$
 - $P(R = heads) = 0.2$
 - $P(R = tails) = 0.8$
- Continuous Example: Blood sugar
 - Variable B , blood sugar value
 - $B \in \mathbb{R}$
 - $P(70 < B < 100) = 0.8$
 - $P(B < 70) = 0.1$
 - $P(B > 100) = 0.1$

Example Discrete Distribution – Categorical and Bernoulli

- A random variable X is said to be Categorically distributed, if it can take on a finite set of n outcomes $\{x_1, \dots, x_k\}$; typically outcomes are just mapped to $\{0, 1, \dots, k - 1\}$ or $\{1, 2, \dots, k\}$.
- We denote it as $X \sim \text{Cat}([p_1, p_2, \dots, p_k])$ where $p_i \in [0,1]$ is the probability of outcome x_i and $\sum_i p_i = 1$
- When all $p_i = \frac{1}{n}$ it is said to be *uniformly* distributed.
- When $n = 2$, this is just the *Bernoulli*(p) distribution with $p \in [0,1]$ and two outcomes $\{0, 1\}$.

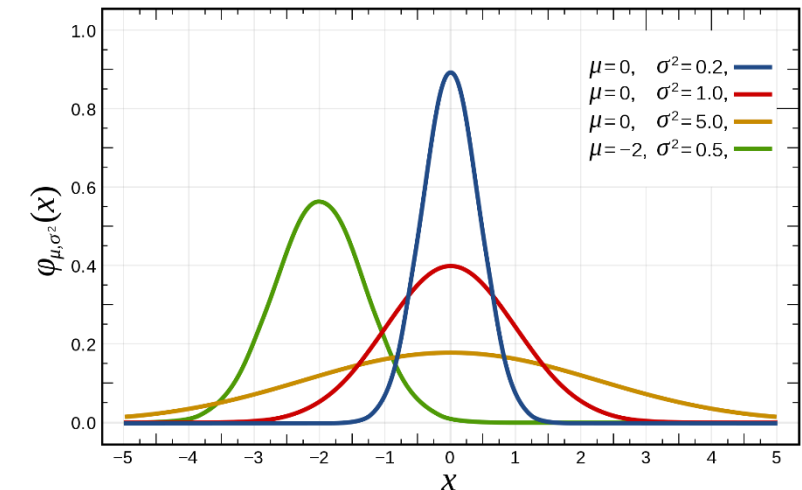
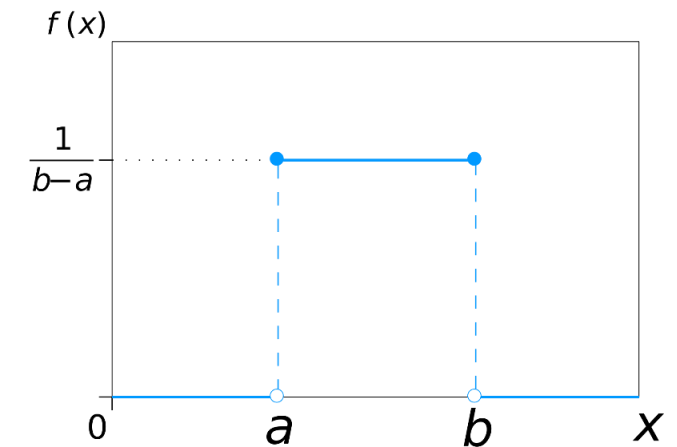
- A process with two outcomes example: flipping a two-sided fair coin
 - $x \in \{0, 1\}$ and $x \sim \text{Bernoulli}(p = 0.5)$
 - Common notation is that $P(x = 1) = p$ and $P(x = 0) = 1 - p$

- A process with 4 outcomes example: pulling a colored ball out of a bag (red, green, blue, yellow)
 - $x \in \{0, 1, 2, 3\}$ and $x \sim \text{Cat}(p = [0.2, 0.3, 0.4, 0.1])$

- **Note:** distributions have parameters. For Cat, the p_i are the parameters. For Bernoulli, the p is the parameter.

Example Continuous Distribution – Uniform and Normal

- Express distribution as a parameterized function of value. This function is called **probability density function** $f(x)$.
- For continuous variables, the $P(v_1 \leq X \leq v_2) = \int_{v_1}^{v_2} f(x)dx$
- It must be the case that: $f(x) \geq 0$ for all x and $\int_{-\infty}^{+\infty} f(x)dx = 1$
- Common continuous distributions:
 - Uniform(a, b); $f(x) = \begin{cases} \frac{1}{b-a} & \text{for } x \in [a, b] \\ 0 & \text{otherwise} \end{cases}$
 - Normal(μ, σ); $f(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{1}{2}\left(\frac{x-\mu}{\sigma}\right)^2}$
- What are parameters of the uniform distribution? What about the normal?



Images from: [link1](#) and [link2](#)

Probability Model and Full-Joint Distribution

- Consider an environment with Temperature and Weather.
- We can construct a **probability model** for this environment using two random variables.
- The **full-joint distribution** over all random variables in a **probability model** gives us all information required to compute any event.

▪ *Joint distribution*

$P(T,W)$

shorthand for P(T and W)

Probability that it's hot AND sunny? $P(T=hot, W=sun)$

Probability that it's hot? $P(T = hot)$

Probability that it's hot OR not foggy? $P(T=hot \text{ OR } W \neq fog)$
 $P(A \vee B) = P(A) + P(B) - P(A \wedge B)$

		Temperature	
		hot	cold
Weather	sun	0.45	0.15
	rain	0.02	0.08
	fog	0.03	0.27
	meteor	0.00	0.00

Marginal Probability

- **Marginal distribution** refers to the probabilities of a subset of variables.
- **Marginalization (summing out)**: is the process of summing out variables from a joint probability.

$$P(X=x) = \sum_y P(X=x, Y=y) \text{ for model with two vars } X, Y$$

$$P(X=x) = \sum_y \sum_z P(X=x, Y=y, Z=z) \text{ with three vars } X, Y, Z$$

$$P(X=x, Y=y) = \sum_z P(X=x, Y=y, Z=z) \text{ with three vars } X, Y, Z$$

- Consider the joint probability table for the temperature and weather model.

What is the probability that it's hot?

$$\begin{aligned} P(T = hot) &= \sum_w P(T = hot, W = w) \\ &= 0.45 + 0.02 + 0.03 + 0.00 = 0.5 \end{aligned}$$

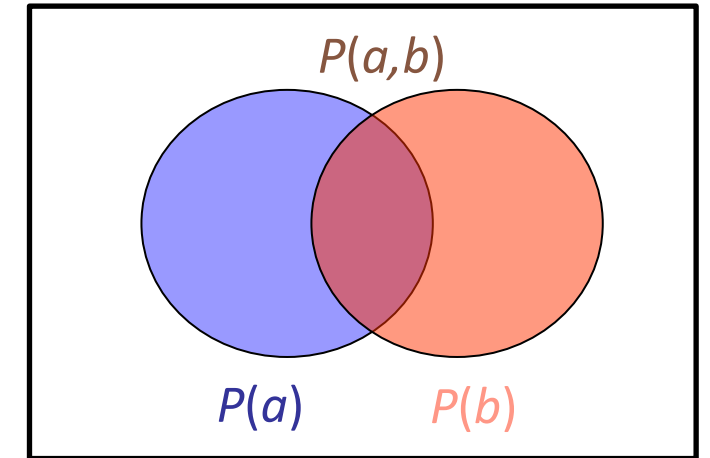
		Temperature		
		hot	cold	
Weather	sun	0.45	0.15	0.60
	rain	0.02	0.08	0.10
	fog	0.03	0.27	0.30
	meteor	0.00	0.00	0.00
		0.50	0.50	

$P(W)$

$P(T)$

Conditional Probability [1]

- $P(A|B)$ is the **conditional** or **posterior** probability of event A, **given** that event B has occurred (B is also referred to as **evidence**).
 - Reflects the updated probability that A is true, after knowing B
e.g. $P(\text{rain in Riyadh tomorrow} \mid \text{raining in Riyadh today})$
- Note that conditional probabilities obey the same rule as non-conditional probabilities.
 - $P(A = a \mid B = b) = 1 - P(A \neq a \mid B = b)$
 - $\sum_{a \in A} P(A = a \mid B = b) = 1$



- A simple relation between joint and conditional probabilities is given by:

$$P(A|B) = \frac{P(A, B)}{P(B)}$$

- Alternatively, for the joint probability (called **Product Rule**):

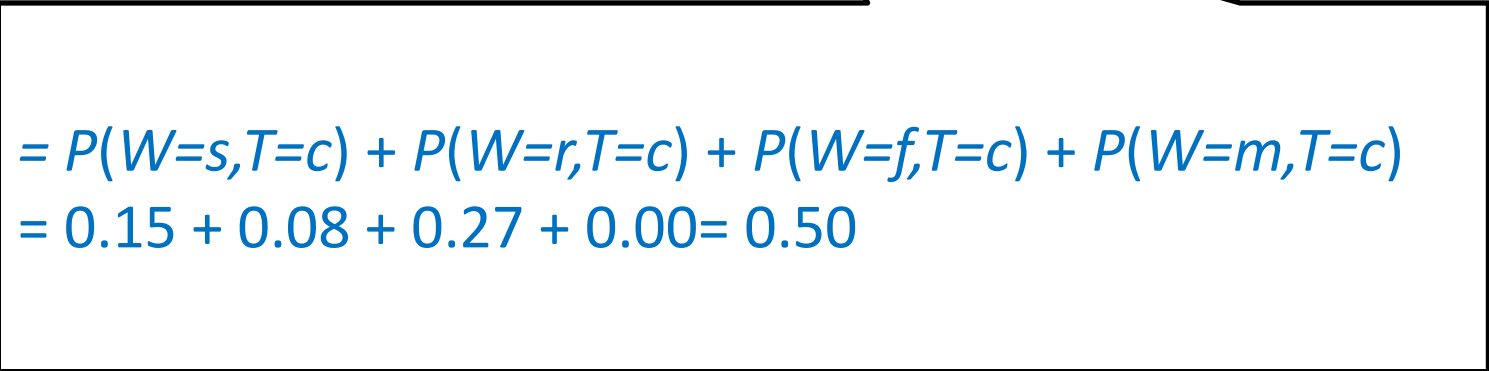
$$P(A, B) = P(A|B)P(B) = P(B|A)P(A)$$

Conditional Probability [2]

- Consider the joint probability table for the temperature and weather model.

What is the probability that it's sunny given that we know it's cold?

$$P(W = s | T = c) = \frac{P(W = s, T = c)}{P(T = c)} = \frac{0.15}{0.50} = 0.3$$


$$\begin{aligned} &= P(W=s, T=c) + P(W=r, T=c) + P(W=f, T=c) + P(W=m, T=c) \\ &= 0.15 + 0.08 + 0.27 + 0.00 = 0.50 \end{aligned}$$

$$P(A|B) = \frac{P(A, B)}{P(B)}$$

		Temperature	
		hot	cold
Weather	sun	0.45	0.15
	rain	0.02	0.08
	fog	0.03	0.27
	meteor	0.00	0.00

Conditional Probability – Quiz

$$P(A|B) = \frac{P(A, B)}{P(B)}$$

$P(X, Y)$

X	Y	P
+x	+y	0.2
+x	-y	0.3
-x	+y	0.4
-x	-y	0.1

■ $P(+x \mid +y) ?$

$$.2/.6=1/3$$

■ $P(-x \mid +y) ?$

$$.4/.6=2/3$$

■ $P(-y \mid +x) ?$

$$.3/.5=.6$$

Chain Rule

- A joint distribution can be written as a product of conditional distributions by **repeated** application of the product rule: $P(A, B) = P(A|B)P(B)$

$$\begin{aligned} P(x_1, x_2, x_3) &= P(x_3|x_1, x_2)P(x_1, x_2) \\ &= P(x_3|x_1, x_2)P(x_2|x_1)P(x_1) \end{aligned}$$

- General form of chain rule:

$$P(x_1, x_2, \dots, x_n) = p(x_1) \prod_{i=2}^n P(x_i|x_1, \dots, x_{i-1})$$

Statistical Modelling and Probabilistic Inference

Statistical Modelling of Environments

- Now that we covered tools from probability, we can look at examples of developing probability models.
- In general, to deal with complex environments with uncertainty, modelling the environment as a probabilistic process helps in decision making and managing risk.
- This involves defining the properties of the environment as random variables following some distribution.
- For example, consider you want to model relationship between weather and temperature as a probabilistic process.
- We have seen a simple model of this earlier in these slides. This model has two variables T and W.
- Discussion: which common distribution is this?
- Let's say we do not make assumptions on the relationship between these two variables, so, we need to define the full-joint probabilities.
- The full-joint table provides complete information with no assumptions. However, there are time complexity issues with this.

$$P(T, W)$$

		Temperature	
		hot	cold
Weather	sun	0.45	0.15
	rain	0.02	0.08
	fog	0.03	0.27
	meteor	0.00	0.00

Model with no assumptions among variables

- Consider an environment with n random variables $x_1, x_2, \dots, x_n$
- No assumptions are made on the relationship among the variables.
- This means when modelling the probability model, it is given by the full-joint:

$$Prob(x_1, x_2, \dots, x_n)$$

- Let's say that each variable is discrete and can take on d values, how many probability entries to store? $O(d^n)$
- So, this is space complexity to store the entries. What about time complexity of answering a query?
- **Probabilistic inference**: compute the probability of a desired event (**query**) using a **probability model**
 - Typically for **query variables given evidence**; $P(Q1, Q2, \dots \mid \text{evidence1, evidence2, } \dots)$
 - E.g., $Prob(\text{airport on time} \mid \text{no accidents}) = 0.90$
 - Represents agent's beliefs given the evidence.
 - Note we can also do inference for query variables without evidence $P(Q1, Q2, \dots)$

Inference by **enumeration** using Full Joint Distribution Table

- Given the full joint distribution table $P(X_1, \dots, X_n)$ we can perform inference by enumeration.
- We want to answer a query: $P(q \mid e)$
- Partition the variables $X_1, \dots, X_n$ into three sets as follows:
 - Query** variables: q
 - Evidence** variables: e
 - Hidden** variables (non-query variables): H

X_1	X_2	...	X_n	$P(X_1, \dots, X_n)$
$X1val1$	$X2val1$	...	$Xnval1$	p_1
$X1val1$	$X2val1$	...	$Xnval2$	p_2
...	...	...	...	...
$X1vald$	$X2vald$	...	$Xnvald$	p_n

$$P(q \mid e) = P(q, e) / P(e)$$

- Step 1: Sum out H from model to get joint of query and evidence. This is summing entries that are consistent with q and e .
- Step 2: In the joint-prob table, find entries consistent with the evidence. Then sum their probas.
- Step 3: Compute result

$$P(q \mid e) = P(q, e) / P(e)$$

$$P(q, e) = \sum_h P(q, e, h)$$

$$P(e) = \sum_x P(e, x)$$

Full-joint inference example

- Use enumeration to compute: $P(W = sun)$?
- This query has no evidence. We proceed by summing out the hidden variables. This is summing entries with (W = sun).

$$P(W = sun) = \sum_{s,t} P(W = sun, S = s, T = t)$$
$$= 0.33 + 0.10 + 0.10 + 0.10 = 0.63$$

Weather	$P(W)$
sun	0.63
rain	0.29
fog	0.08
meteor	0.00

Season	Temp	Weather	P
summer	hot	sun	0.33
summer	hot	rain	0.01
summer	hot	fog	0.01
summer	hot	meteor	0.00
summer	cold	sun	0.10
summer	cold	rain	0.05
summer	cold	fog	0.05
summer	cold	meteor	0.00
winter	hot	sun	0.10
winter	hot	rain	0.03
winter	hot	fog	0.02
winter	hot	meteor	0.00
winter	cold	sun	0.10
winter	cold	rain	0.20
winter	cold	fog	0.00
winter	cold	meteor	0.00

Full-joint inference example

- Use enumeration to compute: $P(W=\text{rain} \mid \text{Season}=\text{winter})$?

$$P(W = \text{rain} \mid S = \text{winter}) = \frac{P(W = \text{rain}, S = \text{winter})}{P(S = \text{winter})}$$
$$= \frac{0.23}{0.45} = 0.51$$

Weather	$P(W \mid S = \text{winter})$
sun	0.45
rain	0.51
fog	0.04
meteor	0.00

Season	Temp	Weather	P
summer	hot	sun	0.33
summer	hot	rain	0.01
summer	hot	fog	0.01
summer	hot	meteor	0.00
summer	cold	sun	0.10
summer	cold	rain	0.05
summer	cold	fog	0.05
summer	cold	meteor	0.00
winter	hot	sun	0.10
winter	hot	rain	0.03
winter	hot	fog	0.02
winter	hot	meteor	0.00
winter	cold	sun	0.10
winter	cold	rain	0.20
winter	cold	fog	0.00
winter	cold	meteor	0.00

Full-joint table parameter estimation

- Consider the Season, Temp, Weather model with the given domains.
- How many parameters does this model have?
- What if you don't know the probabilities? How to estimate it?
- In practice, when you use a model, you are assuming that data is generated according to your model. You gather data from the real-world.
- For this particular model, a sample would be a 3-element vector with values for Season, Temp, Weather.
- Say you have collected m samples for this model.
- How to estimate the parameters?
 - The straightforward method would be $\hat{p}(s, t, w) = \frac{\# \text{ samples with } (s, t, w)}{m}$
 - Other methods we won't discuss (see [here](#) for those interested)
- The more samples you have, the better the estimation.

Season	Temp	Weather	P
summer	hot	sun	?
summer	hot	rain	?
summer	hot	fog	?
summer	hot	meteor	?
summer	cold	sun	?
summer	cold	rain	?
summer	cold	fog	?
summer	cold	meteor	?
winter	hot	sun	?
winter	hot	rain	?
winter	hot	fog	?
winter	hot	meteor	?
winter	cold	sun	?
winter	cold	rain	?
winter	cold	fog	?
winter	cold	meteor	?

Full-joint inference complexity

- Suppose we have a probability model with n variables and d possible values for each variable.
- With no assumptions, model is the full-joint with all variables.
- Space complexity? How many rows does the joint-probability table have
 - $O(d^n)$ to store the full joint distribution table.
- Worst case time complexity to answer query?
 - In general, for a given query with evidence, we need to do summation over hidden variables.
 - Consider one query variable and no evidence $P(X_1 = x_1)$
 - Akin to scanning the entire table to find where $X_1 = x_1$ and sum these entries.
 - In general, can take worst-case $O(d^n)$ to process and sum over the table entries.
- Need other efficient methods for performing inference.
- To get efficiency, we make assumptions to enforce a structure on the model:
 - Independence assumptions.
 - Conditional Independence assumptions and naïve Bayes assumption.
 - Bayesian networks (depends on specific structure; generally exponential).

Independence

- Two variables X and Y are (absolutely) **independent** if

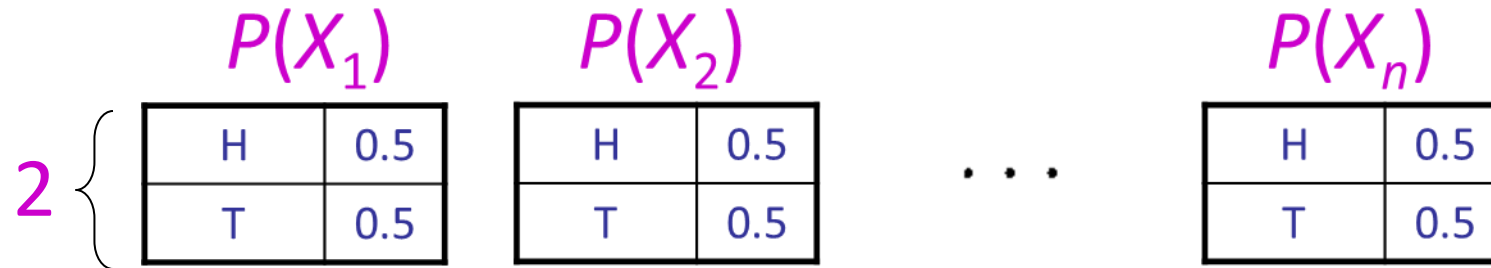
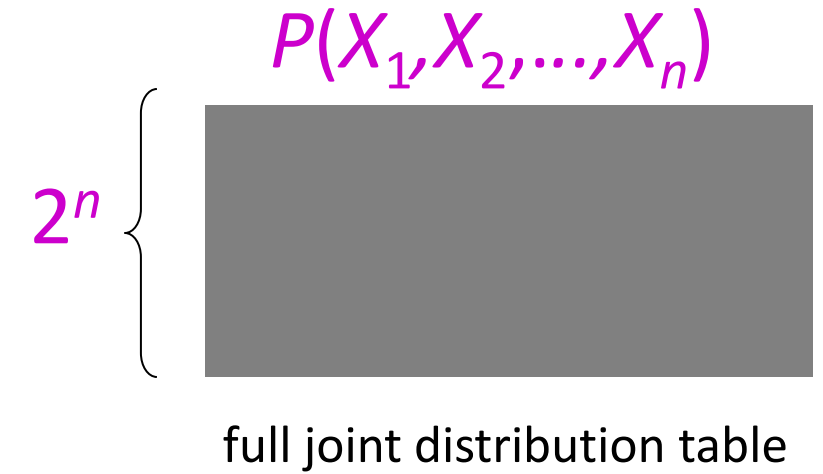
$$\forall x, y \quad P(x, y) = P(x) P(y)$$

- I.e., the joint distribution **factors** into a product of two simpler distributions
- Equivalently, via the product rule $P(x, y) = P(x | y) P(y)$,
 $P(x | y) = P(x)$ or $P(y | x) = P(y)$
- Example: two dice rolls Roll1 and Roll2 that are independent
 - $P(\text{Roll}_1=5, \text{Roll}_2=3) = P(\text{Roll}_1=5) P(\text{Roll}_2=3) = 1/6 \times 1/6 = 1/36$
 - $P(\text{Roll}_2=3 | \text{Roll}_1=5) = P(\text{Roll}_2=3)$

Exploiting Independence

- Suppose we have n fair independent coin flips.
- The full joint distribution table consists of n variables with 2 values each.
- However, suppose they are independent, then:

$$P(X_1, X_2, \dots, X_n) = P(X_1)P(X_2) \dots P(X_n)$$



- For this problem, using independence, our *worst-case* space and time complexity goes from 2^n to $2n$.
- Thus, exploiting independence is one way to reduce complexity of inference.

Conditional Independence and Naïve Bayes

- X is **conditionally independent** of Y given Z if and only if:

$$\forall x, y, z \quad P(x, y \mid z) = P(x \mid z) P(y \mid z)$$

- Also denoted as: $X \perp Y \mid Z$
- Equivalently, if and only if,

$$\forall x, y, z \quad P(x \mid y, z) = P(x \mid z)$$

$$\forall x, y, z \quad P(y \mid x, z) = P(y \mid z)$$

- Can generalize to more than 2 random variables.
 - E.g., k different symptom variables $X_1, X_2, \dots, X_K$, and D = disease, reducing to:

$$P(X_1, X_2, \dots, X_K \mid D) = \prod P(X_i \mid D)$$

- Also known as **naïve Bayes** assumption: a set of variables are all conditionally independent of each other given D.

Exploitation comparison: full-joint table vs independence vs conditional independence

- If we had N **binary** Random Variables $P(X_1, X_2, \dots, X_N)$
 - Full Joint Probability Table (no assumptions)
 - 1 Table with 2^N Rows; $N = 100$, $2^{100} \approx 10^{30}$
 - Assuming variables are independent of each other
 - $P(X_1, X_2, \dots, X_N) = P(X_1)P(X_2) \dots P(X_N)$
 - N tables. Total entries $1 + 1 + \dots + 1 = O(N)$
 - Assuming Naïve Bayes Model (Conditional Independence of $n - 1$ variables given one variable)
 - $P(X_1, X_2, \dots, X_N) = P(X_1)P(X_2, X_3, \dots, X_N \mid X_1) = P(X_1)\prod P(X_i \mid X_1)$
 - N tables. Total entries $1 + 2 + 2 + \dots + 2 = 1 + 2(N - 1) = O(N)$
- Thus, exploiting these independence is one way to reduce complexity of inference.
- Another approach is to impose structure on how the variables influence each other using Bayesian networks.

Bayesian Networks

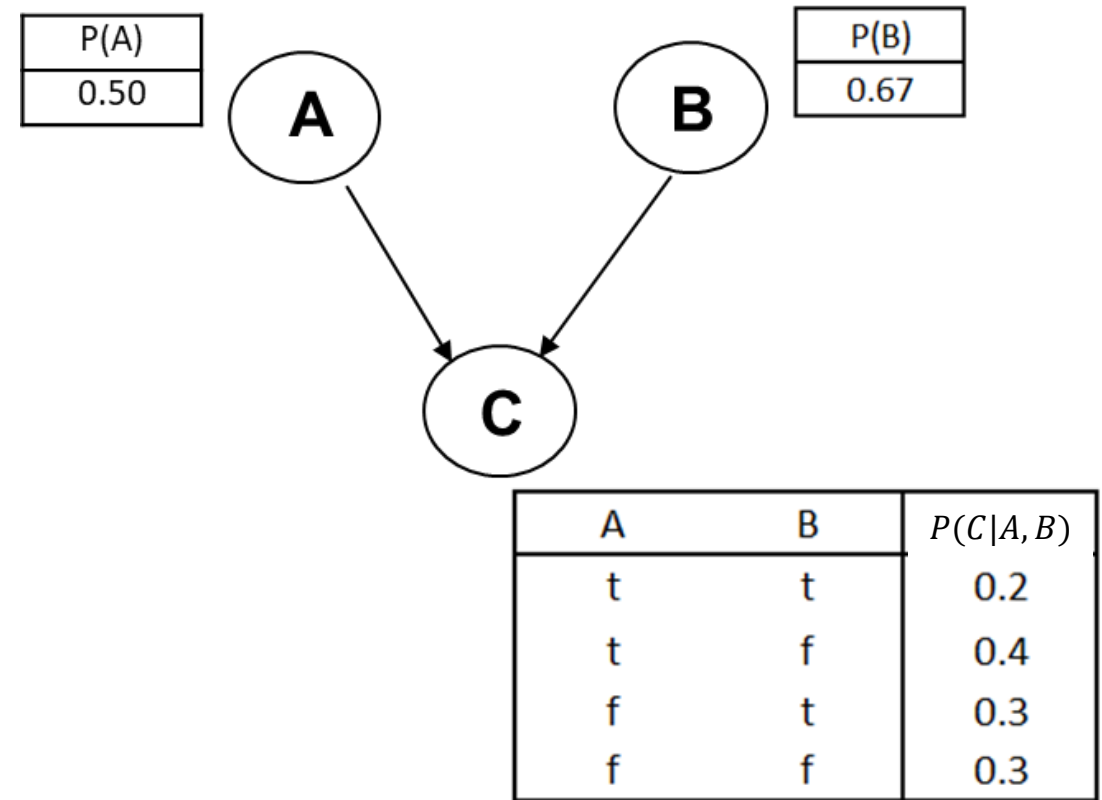
Probabilistic Graphical Model (PGM)

- PGMs: a way to define the structure of variable interaction in a probability model using graphs (nodes and edges)
- Useful for human visualization and understanding of how variables interact, and samples are generated.
- Consider an environment with n variables $X_1, X_2, \dots, X_n$.
 - A PGM $G_1 = (V_1, E_1)$ for this environment would have variables as nodes and relationship between them as edges.
 - Another PGM $G_2 = (V_2, E_2)$ will still have variables as nodes, but edges may be different to represent different interaction.
- Semantically, the presence of an edge between X and Y indicates direct dependence relationship.
- Can be constructed using domain knowledge (handcrafted), but algorithms exist to construct from data.
- Types of PGMs:
 - Undirected PGMs: undirected edges and indicate *correlation* between variables. (for those interested see [link](#))
 - Directed PGMs: directed edges indicate *causality*.
 - Bayesian Networks (Directed Acyclic Graphs): edges are directed, indicate causality, and **no** cycles in graph.
- Our focus will be Bayesian Networks

Bayesian Networks – Overview

- **Bayesian networks**: a technique for describing complex joint distributions (probability models) using simple conditional distributions.
- Idea: Use local causality/conditional independence; the world is composed of many variables, each interacting locally with a few others.
- Provides a way to **compactly** represent the full-joint distribution.
- Bayesian networks are visually represented as a directed graph (nodes and directed edges).

- Bayesian networks are visually represented as a directed graph (nodes and directed edges).
 - Nodes represent random variables.
 - Directed edge represent conditional independence.
- Allows concise view of joint distribution relationships:
 - Nodes and edges show conditional relationships between variables.
 - Conditional distribution represented as a **conditional probability table** (CPT) giving the distribution over variable X for each combination of $Parents(X)$.
- Recall that tables can be **compact/concise**: $P(\neg A)$ does not need to be stored since we can infer $P(\neg A) = 1 - P(A)$.

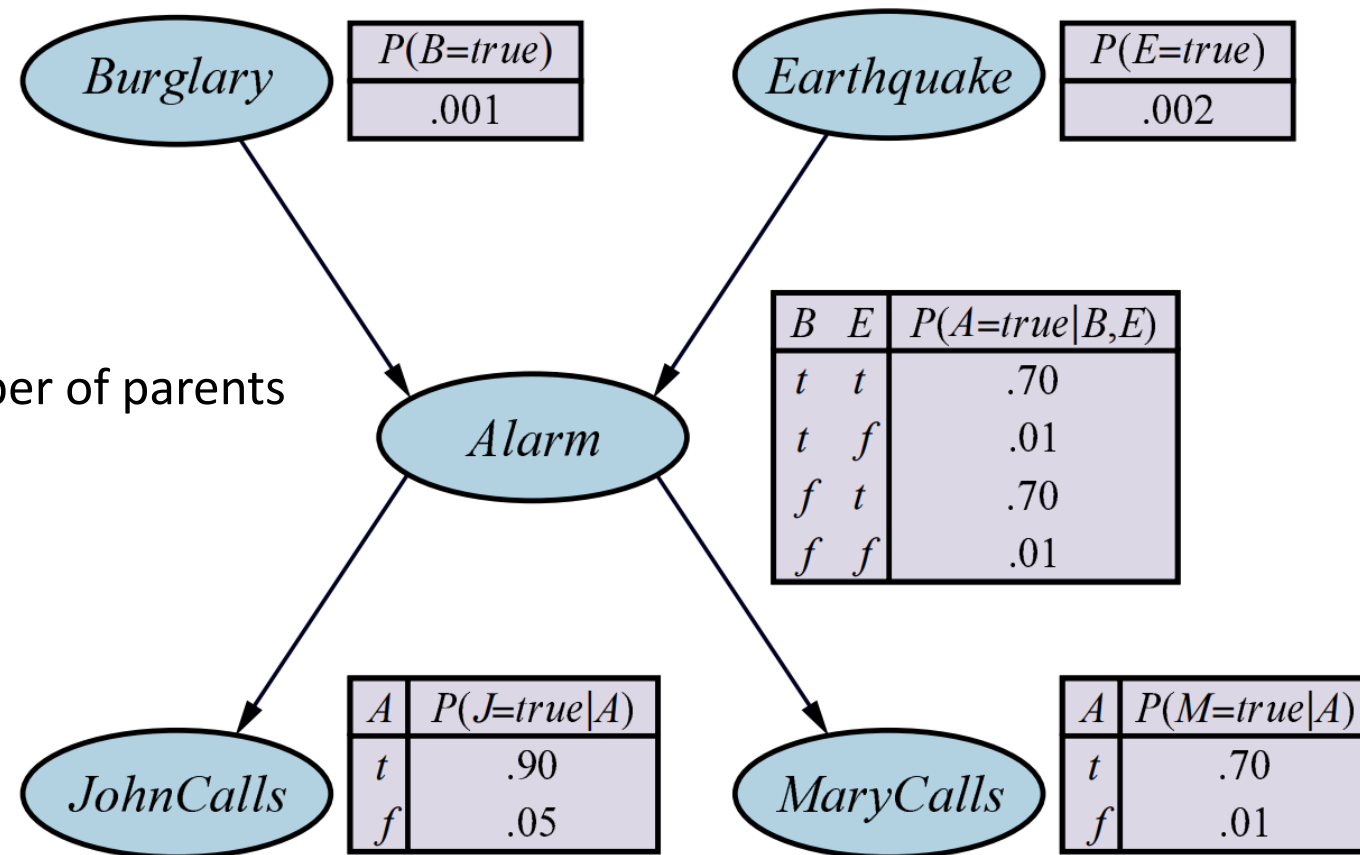


Example of Bayesian network for a model with binary variables A, B, and C. Semantically, the joint probability is given by:

$$P(A, B, C) = P(A)P(B)P(C|A, B)$$

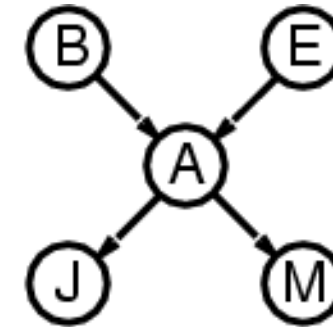
Example: Burglary Network

- I'm at work, neighbor John calls to say my alarm is ringing, but neighbor Mary doesn't call. Sometimes it's set off by minor earthquakes. Is there a burglar?
- Variables: *Burglary*, *Earthquake*, *Alarm*, *JohnCalls*, *MaryCalls*
- Network topology reflects "causal" knowledge:
 - An alarm caused by burglar.
 - An alarm caused by earthquake.
 - MaryCall caused by alarm.
 - JohnCall caused by alarm.
- Number of entries in each CPT? Depends on number of parents
 - Burglar needs 1
 - Earthquake needs 1
 - Alarm needs $2 \times 2 = 4$
 - JohnCalls needs 2
 - MaryCalls needs 2



Compactness

- A CPT for Binary X_i with k Binary parents has 2^k rows for the combinations of parent values.
 - Each row requires one entry for $X_i = \text{true}$
 - If each variable has no more than k parents, the complete network requires $O(n2^k)$ entries.
 - I.e., grows linearly with n , vs. $O(2^n)$ for the full joint distribution.
- For alarm net, $1 + 1 + 4 + 2 + 2 = 10$ numbers (vs. $2^5 - 1 = 31$)
- For a general BN with
 - n variables
 - max of d possible values for each variable
 - Max of k parents for each variable
 - Full joint table has $O(d^n)$ entries
 - **BN space complexity** $O(n d^k)$ entries; linear in n



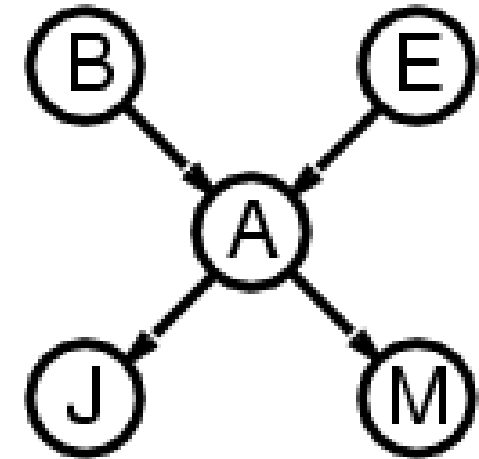
Bayesian Networks – Semantics

- Bayes nets encode joint distributions as product of conditional distributions given parents:

$$P(X_1, X_2, \dots, X_n) = \prod_i P(X_i \mid \text{Parents}(X_i))$$

- For alarm net, what is $P(j \wedge m \wedge a \wedge \neg b \wedge \neg e)$?

$$= P(\neg b) P(\neg e) P(a \mid \neg b, \neg e) P(j \mid a) P(m \mid a)$$



Notice that the network gives the order of sampling
First B and E are sampled, then A is sampled given B and E,
then J is sampled given A, M is sampled given A.

Bayesian Networks – Semantics

- Compare BN semantics:

$$P(X_1, X_2, \dots, X_n) = \prod_i P(X_i \mid Parents(X_i))$$

with the chain rule identity:

$$P(X_1, X_2, \dots, X_n) = \prod_i P(X_i \mid X_1, X_2, \dots, X_{i-1})$$

- So, the Bayes net asserts **conditional independences** $P(X_i \mid X_1, X_2, \dots, X_{i-1}) = P(X_i \mid Parents(X_i))$

- From the conditional independence in BNs, we get two nice properties.
- “Every variable is conditionally independent of its non-descendants given its parents”
- “a node is conditionally independent of all other nodes in the network, given its parents, children, and children’s parents—that is, given its **Markov blanket**”

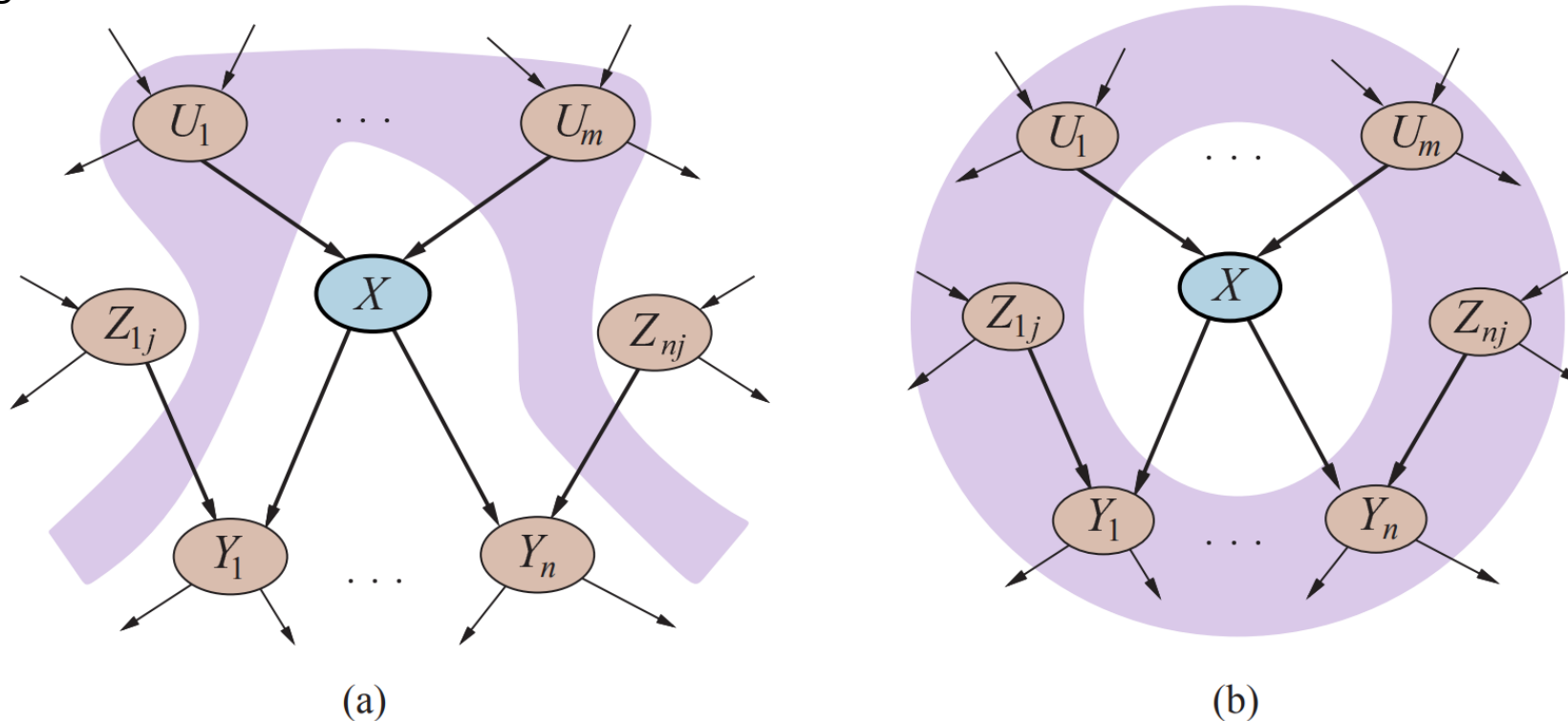
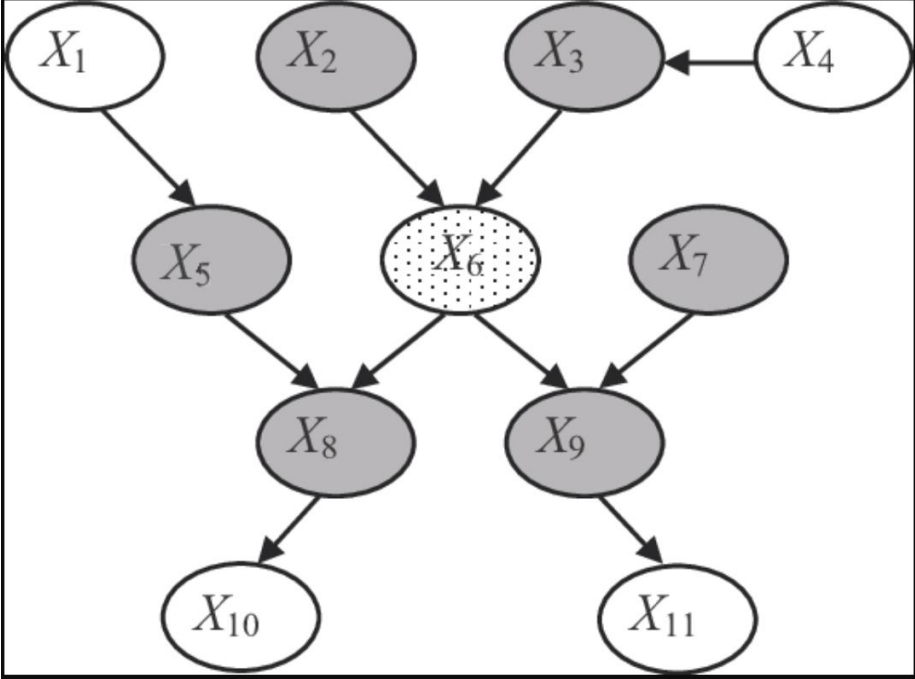


Figure 13.4 (a) A node X is conditionally independent of its non-descendants (e.g., the Z_{ij} s) given its parents (the U_i s shown in the gray area). (b) A node X is conditionally independent of all other nodes in the network given its Markov blanket (the gray area).

“Every variable is conditionally independent of its non-descendants given its parents” Example

What is $P(x_5, x_6 \mid x_2, x_3)$?



$$\begin{aligned}
 P(x_5, x_6 \mid x_2, x_3) &= \frac{P(x_5, x_6, x_2, x_3)}{P(x_2, x_3)} \\
 &= \frac{\sum_{X_1, X_4, X_7, X_8, X_9, X_{10}, X_{11}} P(X_1, x_2, x_3, X_4, x_5, x_6, X_7, X_8, X_9, X_{10}, X_{11})}{\sum_{X_1, X_4, X_5, X_6, X_7, X_8, X_9, X_{10}, X_{11}} P(X_1, x_2, x_3, X_4, X_5, X_6, X_7, X_8, X_9, X_{10}, X_{11})} \\
 &= \frac{\sum_{X_1, X_4, X_7, X_8, X_9, X_{10}, X_{11}} P(X_1)P(x_5 \mid X_1)P(X_4)P(x_3 \mid X_4)P(X_7)P(x_2)P(x_6 \mid x_2, x_3)P(X_9 \mid x_6, X_7)P(X_8 \mid x_5, x_6)P(X_{10} \mid X_8)P(X_{11} \mid X_9)}{\sum_{X_1, X_4, X_5, X_6, X_7, X_8, X_9, X_{10}, X_{11}} P(X_1)P(X_5 \mid X_1)P(X_4)P(x_3 \mid X_4)P(X_7)P(x_2)P(X_6 \mid x_2, x_3)P(X_9 \mid X_6, X_7)P(X_8 \mid X_5, X_6)P(X_{10} \mid X_8)P(X_{11} \mid X_9)}
 \end{aligned}$$

Some terms here sum to 1, and some terms cancel with denominator.

$$= P(x_6 \mid x_2, x_3) \frac{\sum_{X_1} P(X_1)P(x_5 \mid X_1)}{\sum_{X_1, X_5} P(X_1)P(X_5 \mid X_1)}$$

Denominator sums to 1

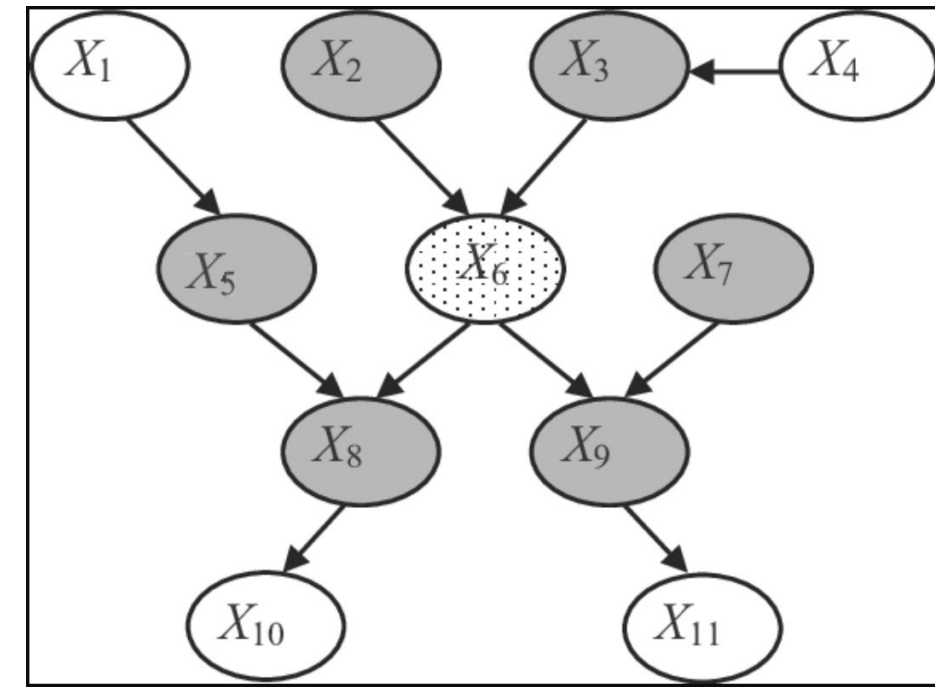
$$= P(x_6 \mid x_2, x_3)P(x_5) = P(x_6 \mid x_2, x_3)P(x_5 \mid x_2, x_3)$$

So, we could have just decoupled x5 and x6 from the beginning using the rule.

“a node is conditionally independent of all other nodes in the network, given its parents, children, and children’s parents—that is, given its **Markov blanket**”

What is $P(x_6 \mid x_2, x_3, x_5, x_7, x_8, x_9)$? **given the Markov blanket of X_6**

$$\begin{aligned}
 P(x_6 \mid x_2, x_3, x_5, x_7, x_8, x_9) &= \frac{P(x_6, x_2, x_3, x_5, x_7, x_8, x_9)}{P(x_2, x_3, x_5, x_7, x_8, x_9)} \\
 &= \frac{\sum_{X_1, X_4, X_{10}, X_{11}} P(X_1, X_4, X_{10}, X_{11}, x_6, x_2, x_3, x_5, x_7, x_8, x_9)}{\sum_{X_1, X_4, X_6, X_{10}, X_{11}} P(X_1, X_4, X_6, X_{10}, X_{11}, x_2, x_3, x_5, x_7, x_8, x_9)} \\
 &= \frac{\sum_{X_1, X_4, X_{10}, X_{11}} P(X_1)P(x_5 \mid X_1)P(X_4)P(x_3 \mid X_4)P(x_7)P(x_2)P(x_6 \mid x_2, x_3)P(x_9 \mid x_6, x_7)P(x_8 \mid x_5, x_6)P(X_{10} \mid x_8)P(X_{11} \mid x_9)}{\sum_{X_1, X_4, X_6, X_{10}, X_{11}} P(X_1)P(x_5 \mid X_1)P(X_4)P(x_3 \mid X_4)P(x_7)P(x_2)P(X_6 \mid x_2, x_3)P(x_9 \mid X_6, x_7)P(x_8 \mid x_5, X_6)P(X_{10} \mid x_8)P(X_{11} \mid x_9)} \\
 &= \frac{P(x_6 \mid x_2, x_3)P(x_9 \mid x_6, x_7)P(x_8 \mid x_5, x_6)}{\sum_{X_6} P(X_6 \mid x_2, x_3)P(x_9 \mid X_6, x_7)P(x_8 \mid x_5, X_6)}
 \end{aligned}$$



Notice that given the Markov blanket of X_6 , we do not need to involve other variables to compute the query.

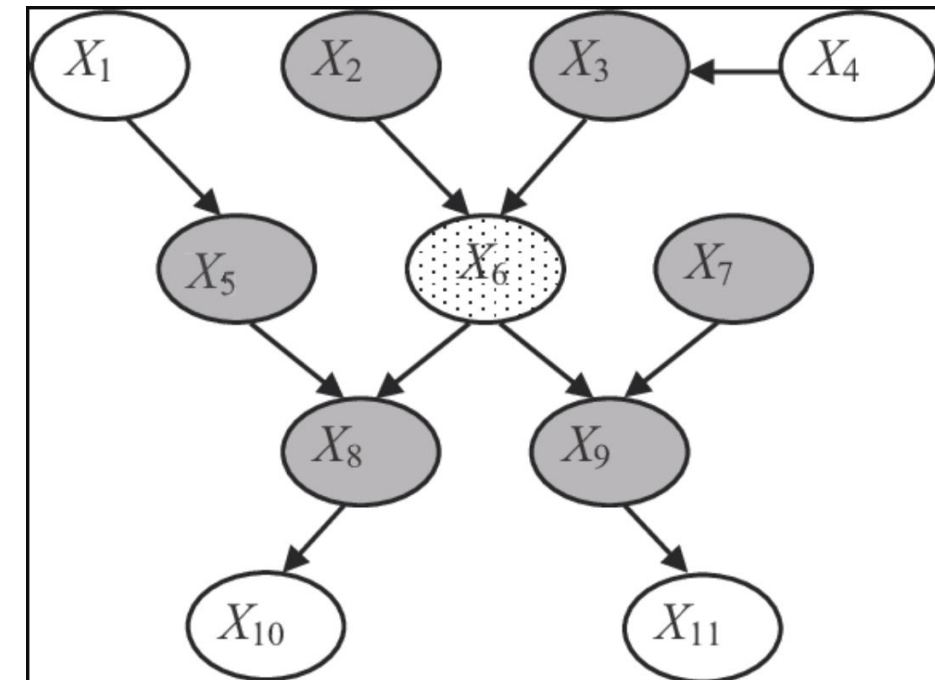
What is $P(x_{11}, x_6 \mid x_2, x_3, x_5, x_7, x_8, x_9)$?

Using Markov blanket for x6 is given

$$= P(x_6 \mid x_2, x_3, x_5, x_7, x_8, x_9) P(x_{11} \mid x_2, x_3, x_5, x_7, x_8, x_9)$$

Using that parents of x11 are given

$$= P(x_6 \mid x_2, x_3, x_5, x_7, x_8, x_9) P(x_{11} \mid x_9)$$



- X is **conditionally independent** of Y given Z if and only if:

$$\forall x, y, z \quad P(x, y \mid z) = P(x \mid z) P(y \mid z)$$
- Also denoted as: $X \perp Y \mid Z$
- Equivalently, if and only if,

$$\forall x, y, z \quad P(x \mid y, z) = P(x \mid z)$$

$$\forall x, y, z \quad P(y \mid x, z) = P(y \mid z)$$

- Can use these two properties to decouple queries given evidence.
- There is a more formal algorithm/process to determine which variables are independent given evidence in a Bayesian network called **d-separation**.

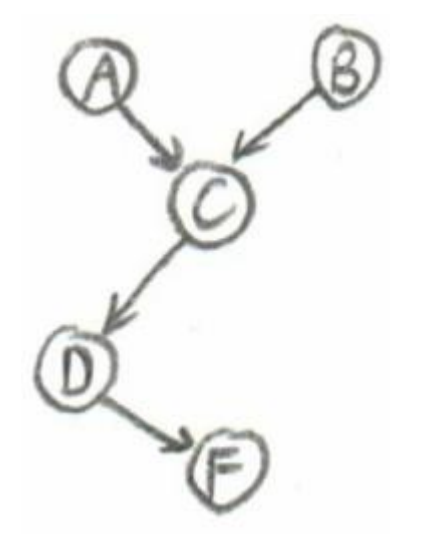
D-Separation – technique to determine independence in a BN

- There is a more formal algorithm/process to determine which variables are independent given evidence in a Bayesian network called **d-separation**.
- Given a query $P(Q \mid e)$, D-separation works as follows:
 1. Draw the ancestral graph. Construct the “ancestral graph” of all variables mentioned in the probability expression (Q and e). This is a reduced version of the original network, consisting only of the variables mentioned and all of their ancestors (parents, parents’ parents, etc.)
 2. Connect parents. For each pair of variables with a common child, draw an undirected edge (line) between them. (If a variable has more than two parents, draw lines between every pair of parents)
 3. Replace the directed edges (arrows) with undirected edges (lines).
 4. Delete the given variables e and their edges.
 5. Determine independence using available paths:
 - *Disconnected* variables in this graph, are guaranteed to be independent given the evidence.
 - *Connected* variables in this graph are **not** guaranteed to be independent given the evidence.
- See <http://web.mit.edu/jmn/www/6.034/d-separation.pdf> for a nice explanation and examples.

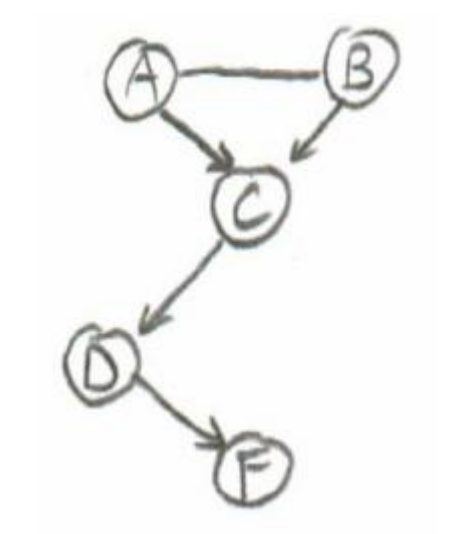
BN D-Separation Example [1]

Are A and B conditionally independent given D and F? $A \perp\!\!\!\perp B \mid D, F$

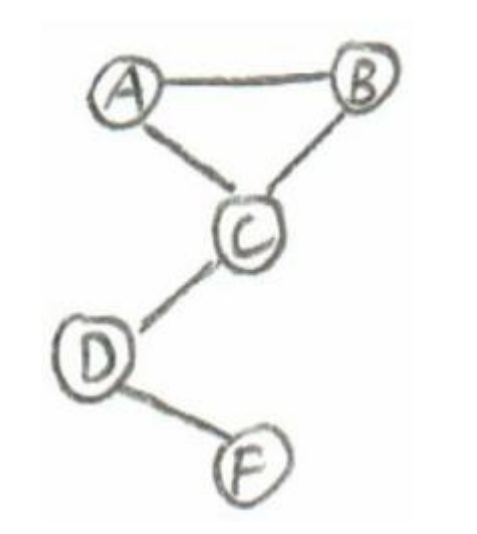
Step 1: ancestry graph



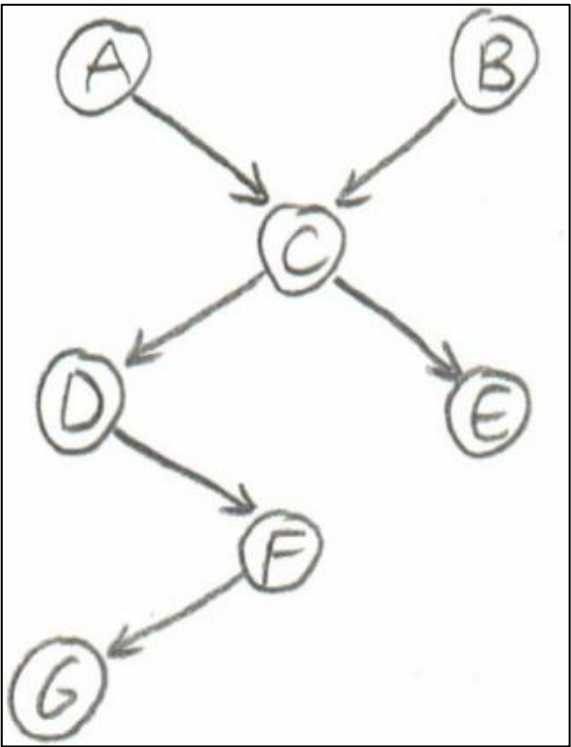
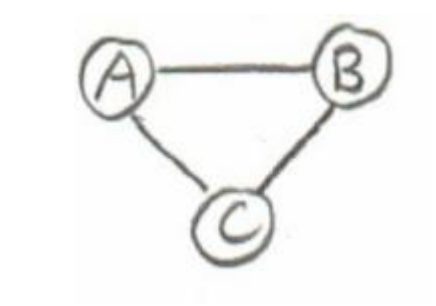
Step 2: connect parents



Step 3: make undirected



Step 4: delete givens



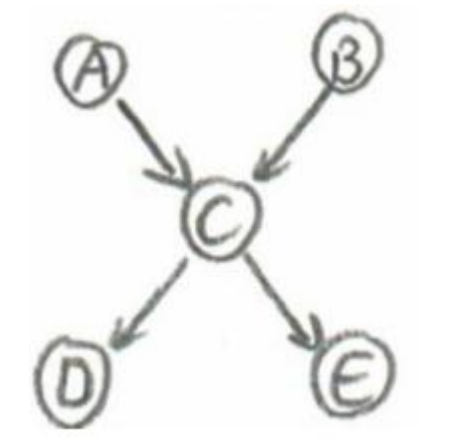
Bayesian network

Answer: There is a path between A and B, so we cannot confirm the statement.

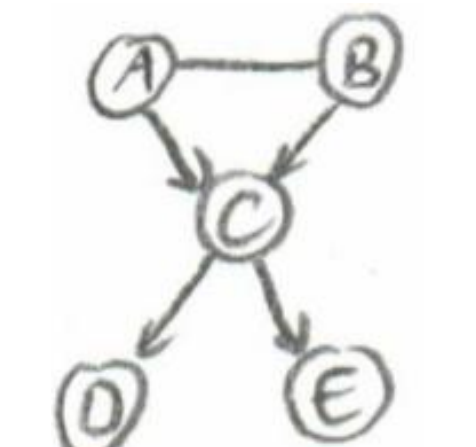
BN D-Separation Example [2]

Is $D \perp\!\!\!\perp E \mid C$? Note that $\perp\!\!\!\perp$ is independence symbol.

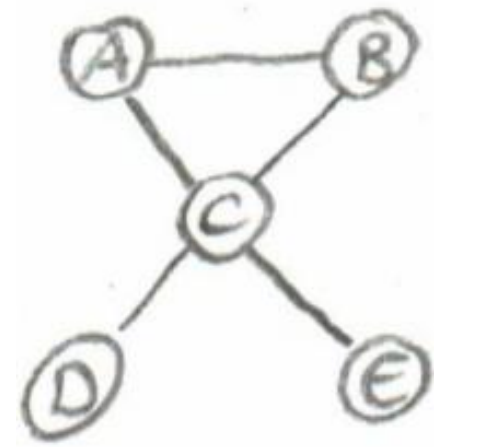
Step 1: ancestry graph



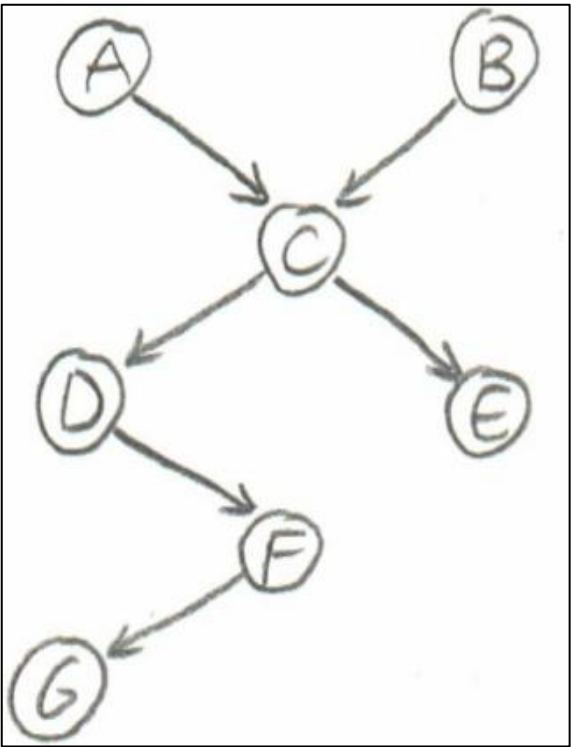
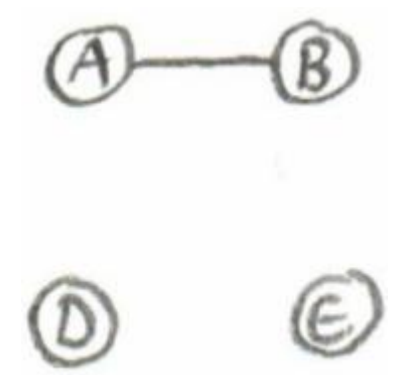
Step 2: connect parents



Step 3: make undirected



Step 4: delete givens



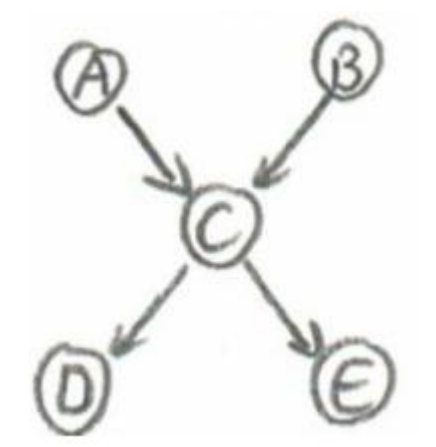
Bayesian network

Answer: There is no path between E and D, so we confirm the statement is true (independent given evidence).

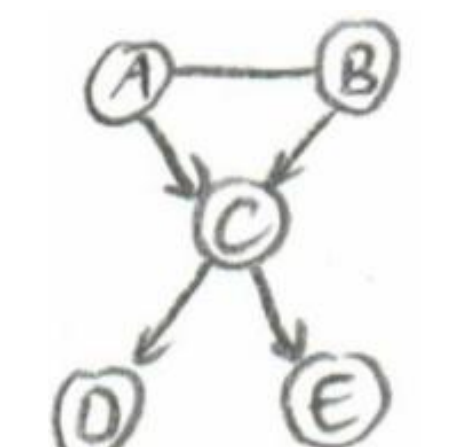
BN D-Separation Example [3]

Is $D \perp\!\!\!\perp E$? Note no evidence given.

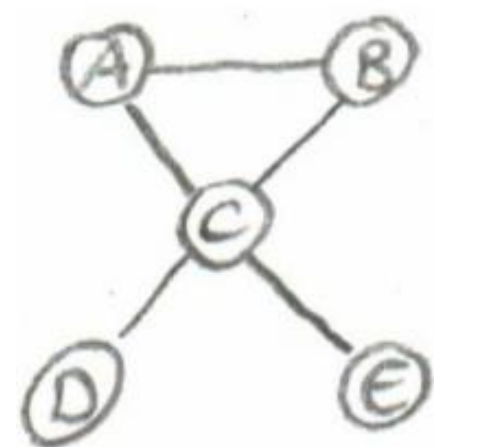
Step 1: ancestry graph



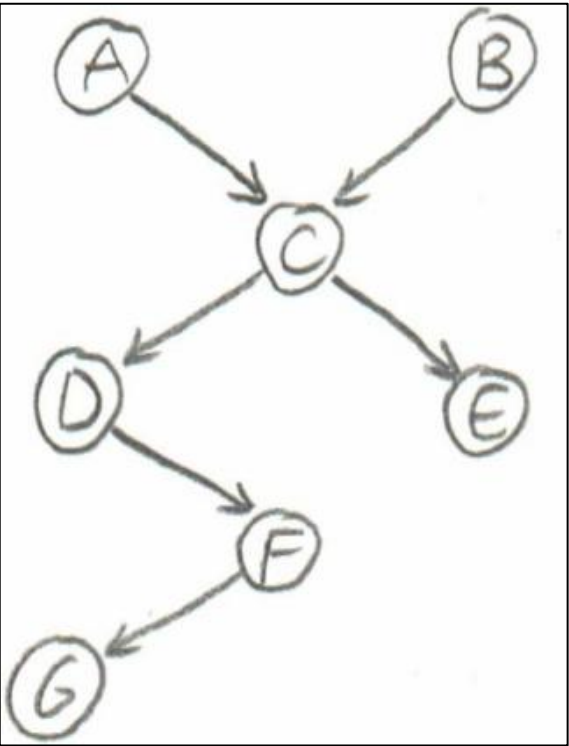
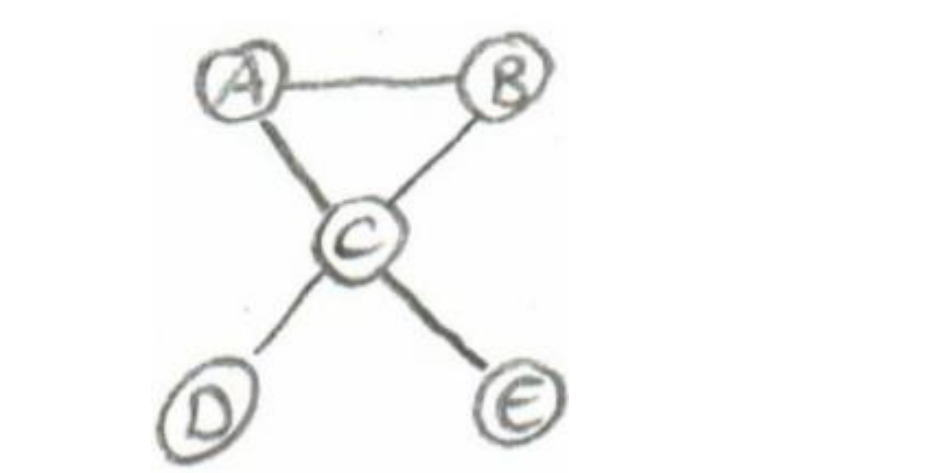
Step 2: connect parents



Step 3: make undirected



Step 4: delete givens, no givens so no deletions



Bayesian network

Answer: There is a path between E and D, so we cannot confirm the statement.

What is $P(j)$?

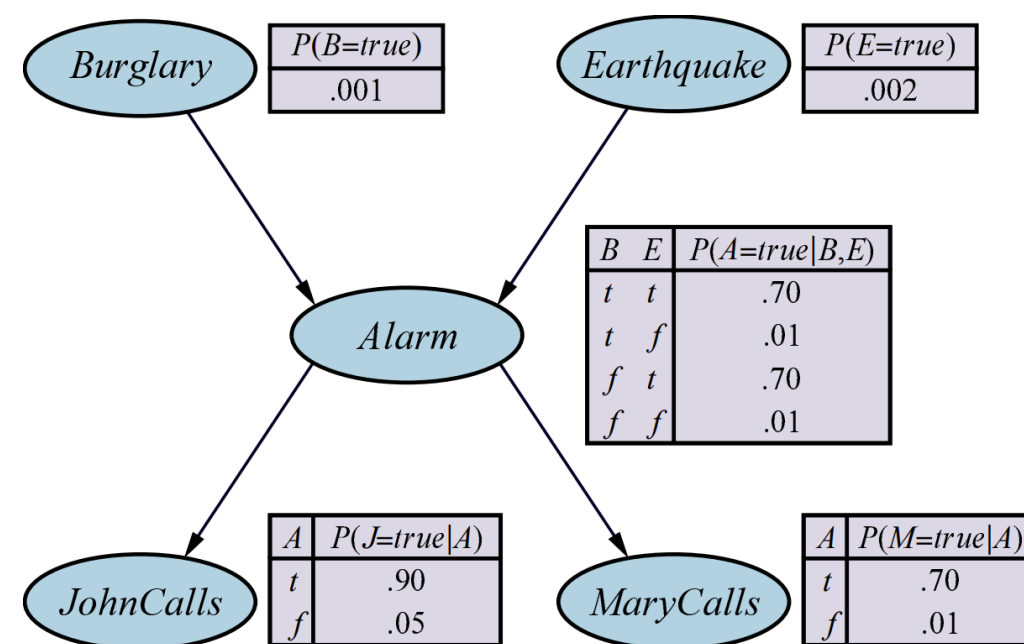
$$P(j) = \sum_{B,E,A,M} P(B, E, A, j, M)$$

$$= \sum_{B,E,A,M} P(B)P(E)P(A | B, E)P(j | A)P(M | A)$$

$$= \sum_B P(B) \left[\sum_E P(E) \left[\sum_A P(A | B, E) P(j | A) \left[\sum_M P(M | A) \right] \right] \right]$$

Sums to 1

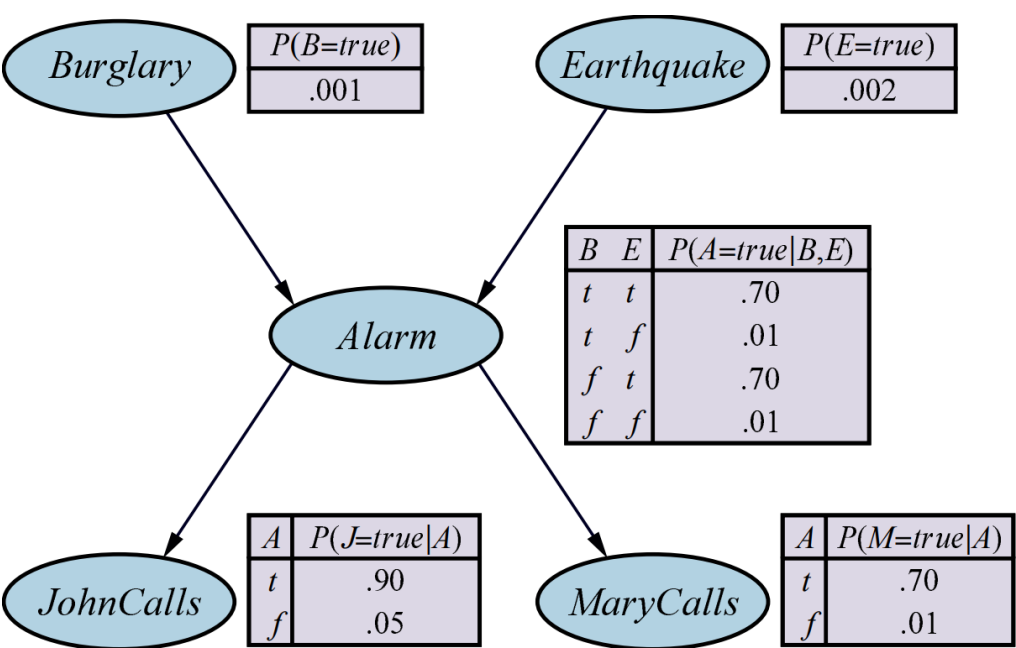
$$= \sum_B P(B) \left[\sum_E P(E) \left[\sum_A P(A | B, E) P(j | A) \right] \right]$$



With a BN, to compute $P(X)$, we just need to sum over the hidden variables that influence X **directly or indirectly**.
 J is influenced by A, B, E . In other words, what variables do I need to know in order to sample J .

What is $P(j, b, e, m \mid a)$?

$$\begin{aligned} P(j, b, e, m \mid a) &= \frac{P(j, b, e, m, a)}{P(a)} \\ &= \frac{P(b)P(e)P(a \mid b, e)P(j \mid a)P(m \mid a)}{\sum_{B, E, J, M} P(B, E, a, J, M)} \\ &= \frac{P(b)P(e)P(a \mid b, e)P(j \mid a)P(m \mid a)}{\sum_{B, E, J, M} P(B)P(E)P(a \mid B, E)P(J \mid a)P(M \mid a)} \\ &= \frac{P(b)P(e)P(a \mid b, e)P(j \mid a)P(m \mid a)}{\sum_{B, E} P(B)P(E)P(a \mid B, E) \underbrace{\sum_J P(J \mid a)}_{\text{Sums to 1}} \underbrace{\sum_M P(M \mid a)}_{\text{Sums to 1}}} \\ &= \frac{P(b)P(e)P(a \mid b, e)P(j \mid a)P(m \mid a)}{\sum_{B, E} P(B)P(E)P(a \mid B, E)} \end{aligned}$$

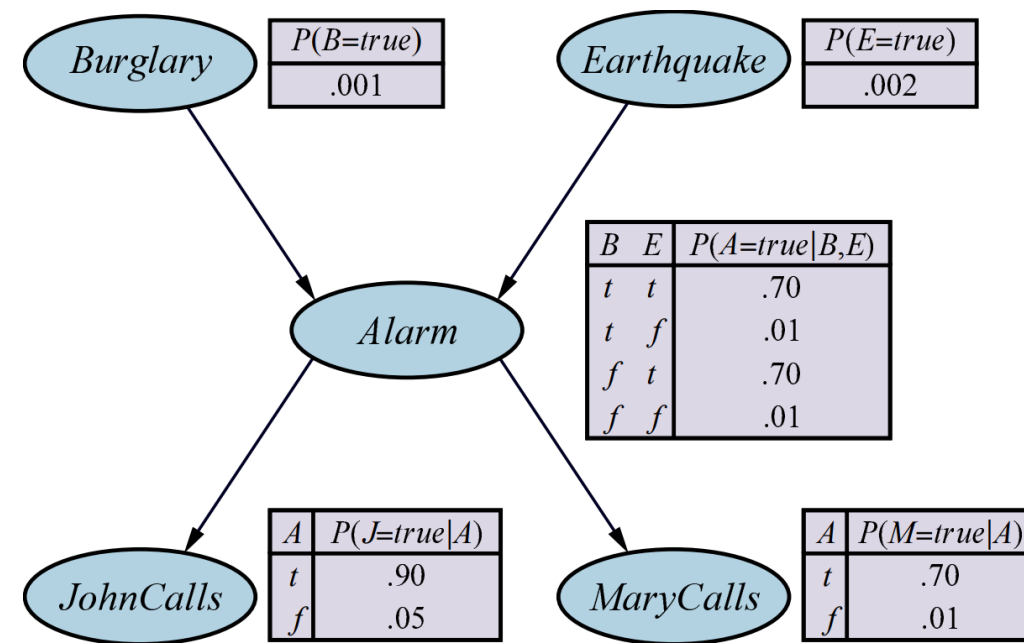


To compute $P(a)$, what variables do I need to know in order to sample A ? Just B and E .

What is $P(\neg b, \neg m \mid a, \neg e)$?

$$P(\neg b, \neg m \mid a, \neg e) = \frac{P(\neg b, \neg m, a, \neg e)}{P(a, \neg e)}$$

$$= \frac{P(\neg b)P(\neg e)P(a \mid \neg b, \neg e)P(\neg m \mid a)}{\sum_B P(B)P(\neg e)P(a \mid B, \neg e)}$$

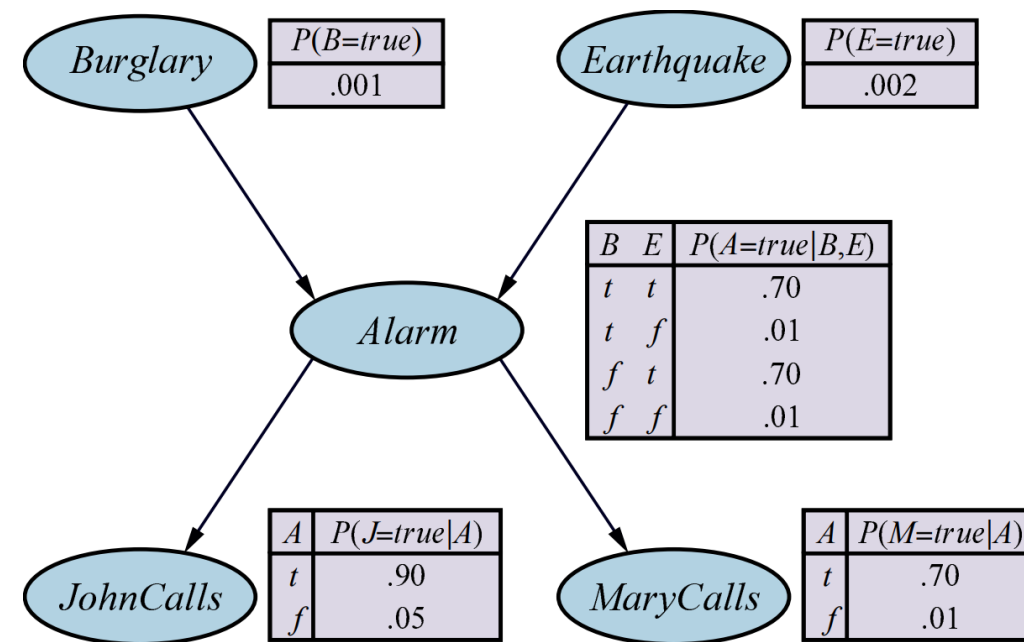


What is $P(a \mid b, e, m, j)$?

$$P(a \mid b, e, m, j) = \frac{P(a, b, e, m, j)}{P(b, e, m, j)}$$

$$= \frac{P(b)P(e)P(a \mid b, e)P(m \mid a)P(j \mid a)}{\sum_A P(b)P(e)P(A \mid b, e)P(m \mid A)P(j \mid A)}$$

$$= \frac{P(a \mid b, e)P(m \mid a)P(j \mid a)}{\sum_A P(A \mid b, e)P(m \mid A)P(j \mid A)}$$



See this link for BN playground

http://www.cs.man.ac.uk/~gbrown/bayes_nets/#

Pgmpy python package

- [pgmpy](#) python package for Bayesian networks.
- It is a package for Bayesian networks. Many assumptions can be modelled as a Bayesian network.
- You can provide the BN structure yourself or use structure-learning algorithms given a dataset.
- You can provide the probabilities yourself or use estimation algorithms given a dataset.
- Supports Exact inference and Approximate (Sampling Based).

Navigation

[Getting Started](#)

[Base Model Structures](#)

[Models](#)

[Parameterization](#)

[Exact Inference](#)

[Model Testing](#)

[Approximate Inference](#)

[Parameter Estimation](#)

[Structure Learning](#)

[Metrics for testing models](#)

[Reading/Writing to File](#)

[Plotting Models](#)

[Example Notebooks](#)

[Tutorial Notebooks](#)

Notes on constructing a Bayesian Network

- How many graphs possible for n variables? How to find the **optimal** graph that best represents the process?
- You can construct the network using an ordering on the variables such that causes precede effects. This will give a more compact network.
- However, in general, finding **optimal** (most compact) Bayesian network for a given problem is NP-hard (factorial and exponential terms).
- Domain knowledge can be used to construct the network directly. Or to give a better ordering of the variables that reflect causal relationships.
- In practice, we can use algorithms guided by heuristics and dataset to construct reasonably compact networks.
- We won't cover these in this course.
- For those interested, see the structure-learning algorithms at <https://pgmpy.org/>

Structure Learning
PC with variants
Hill-Climb Search
Tree Search
Max-Min Hill-Climb
Exhaustive Search

Constructing a Bayesian Network – Impact of node ordering on size of network

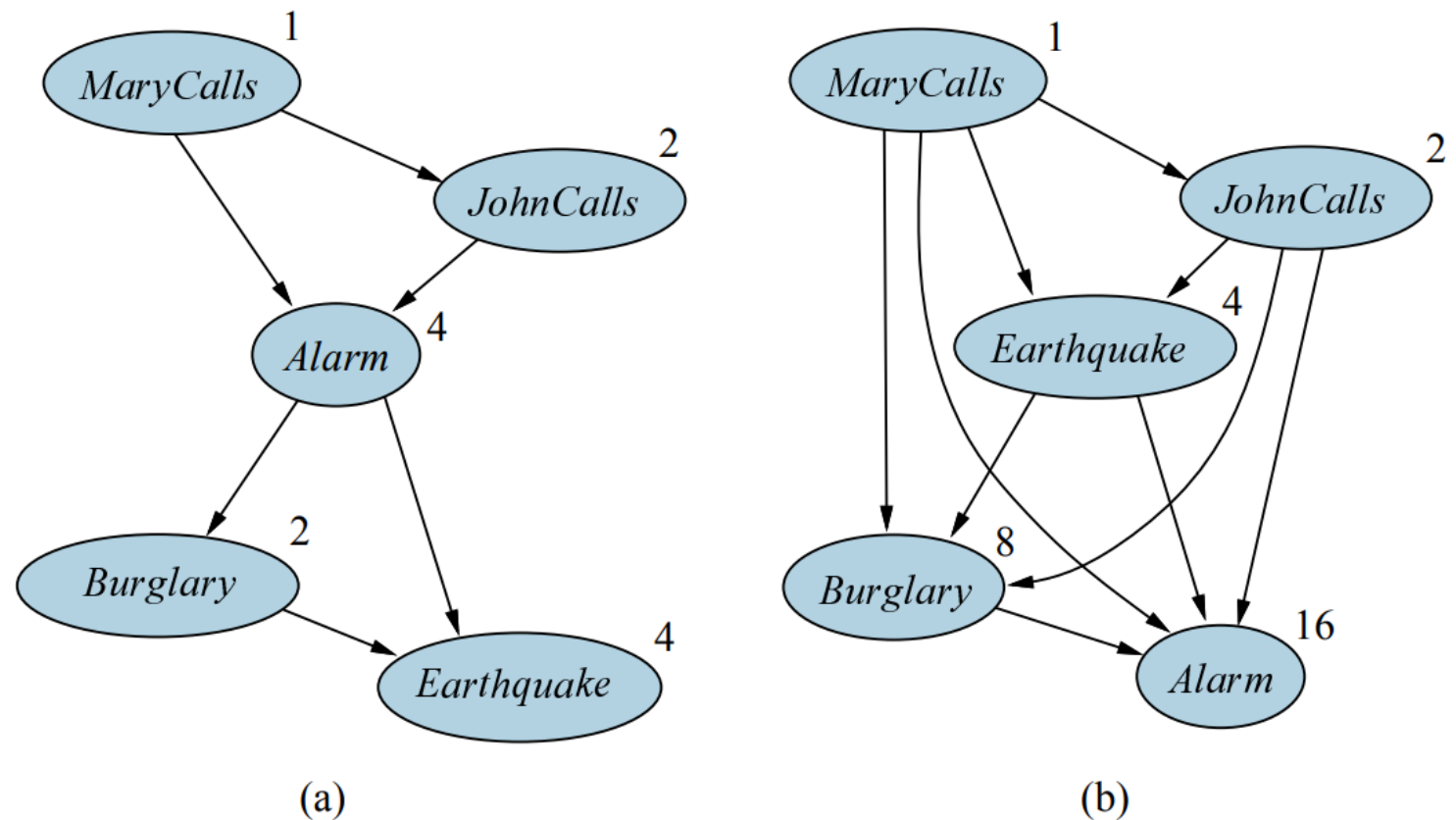
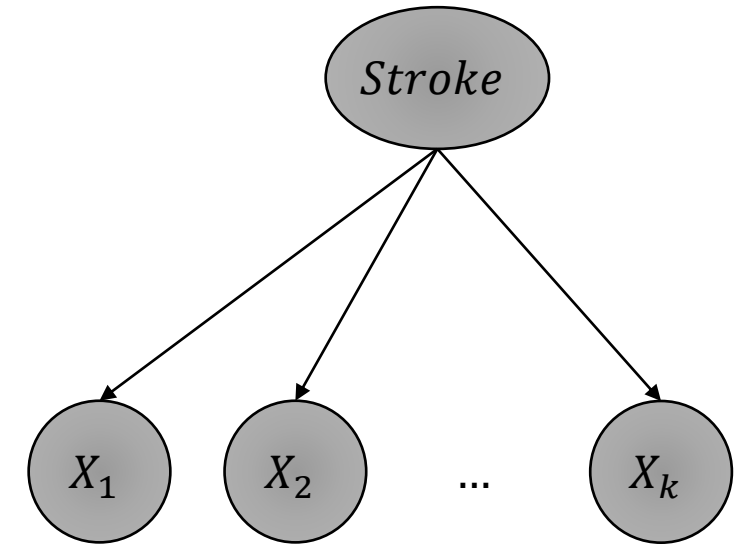


Figure 13.3 Network structure and number of parameters depends on order of introduction. (a) The structure obtained with ordering M, J, A, B, E . (b) The structure obtained with M, J, E, B, A . Each node is annotated with the number of parameters required; 13 in all for (a) and 31 for (b).

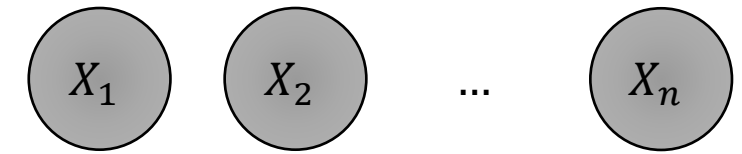
Using data to estimate BN parameters and structure.

- You are given a heart-stroke dataset.
- Let's use a naïve bayes BN to model heart-stroke.
- We can use [pgmpy](#) to learn the BN parameters (probabilities) using the dataset.
- We can use algorithms guided by heuristics and dataset to construct reasonably compact networks. (won't cover these algorithms)
- How to compare between various BN structures for particular problem? Use a performance metric to compare between them. For example, we could have a held-aside heart-stroke set, and see which BN structure did the best in predicting stroke on this set.

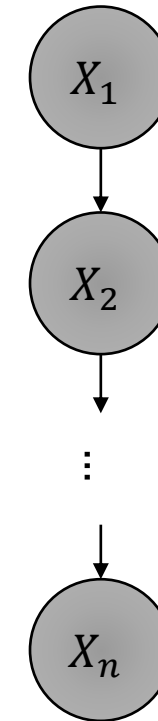


BN structure of naïve Bayes

- Suppose we have n binary variables.
- With no assumptions, we have the full-joint table with $O(2^n)$ entries. This is a BN that is completely connected (all nodes are connected to all other nodes).
- If we assume all variables are independent of each other, this is a BN with no edges (all nodes have no connections).
- If we assume that X_i is influenced by X_{i-1} , we get a tree like BN.
- So, a BN structure is determined by the assumptions we make about the variable relationships. Typically, this is decided on by domain knowledge and intuition about the problem.



BN using all independence assumption.

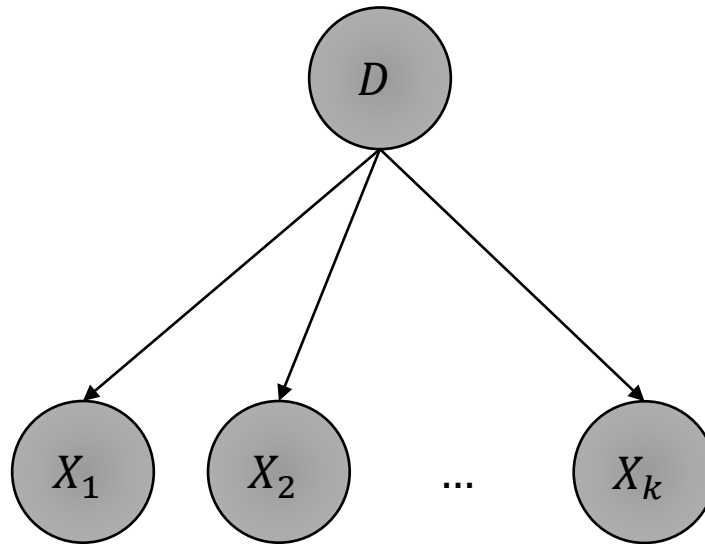


BN with assumption that X_i influenced by X_{i-1}

BN of Naïve Bayes

- Consider a naïve bayes where k different symptom variables $X_1, X_2, \dots, X_k$ are conditionally independent of each other given $D = \text{disease}$.

$$P(X_1, X_2, \dots, X_k \mid D) = \prod P(X_i \mid D)$$



BN structure of naïve Bayes

Example Bayes' Net: Car Insurance

- See textbook section 13.2.4.
- Gives example on how Bayes net is structured for a car insurance application.
- Can ask queries like what is the probability that medical cost will be high given information on the client.
- Typically in practice, the structure of a Bayesian network is determined by the designer using domain knowledge and intuition of what variables influence other variables.

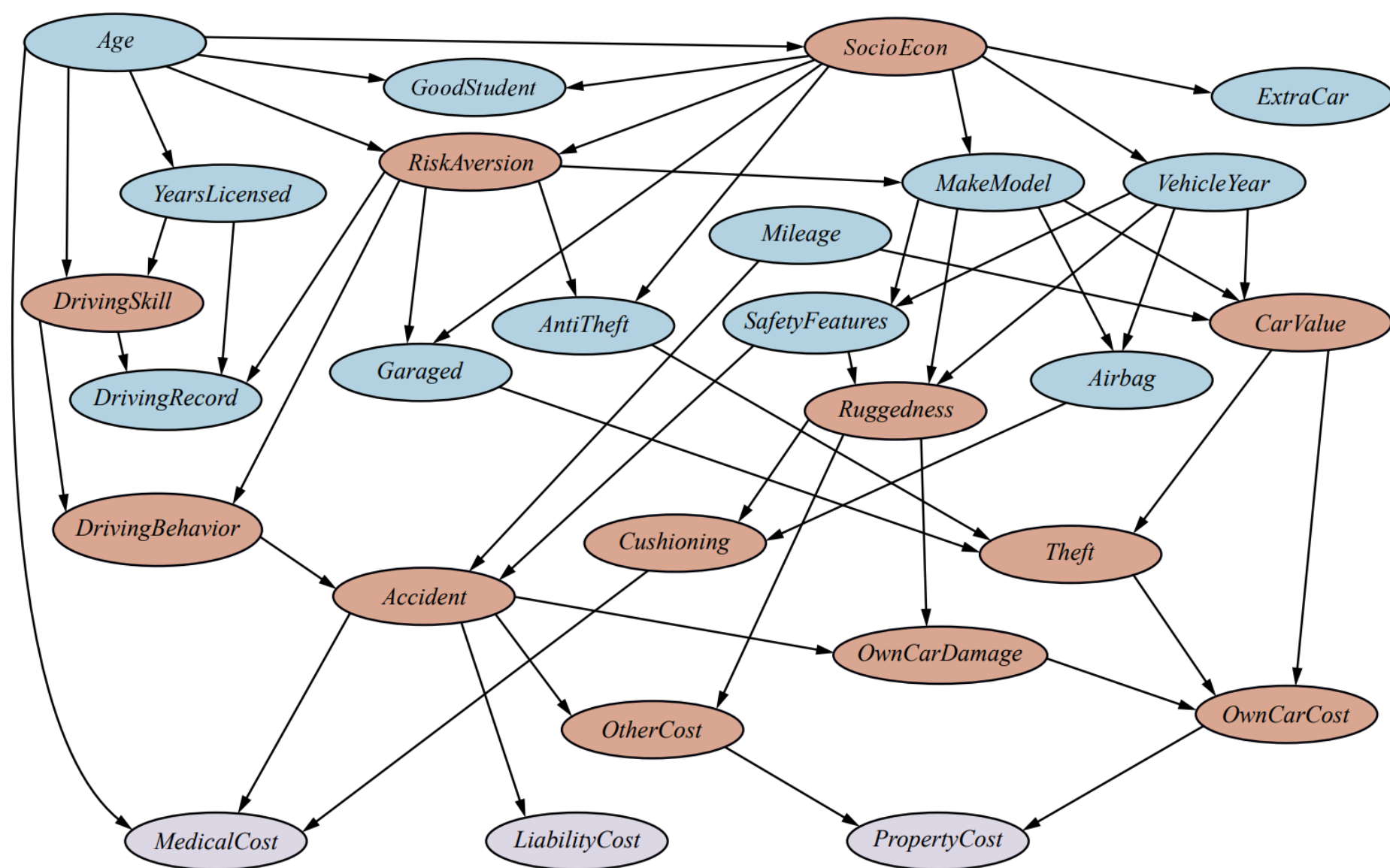
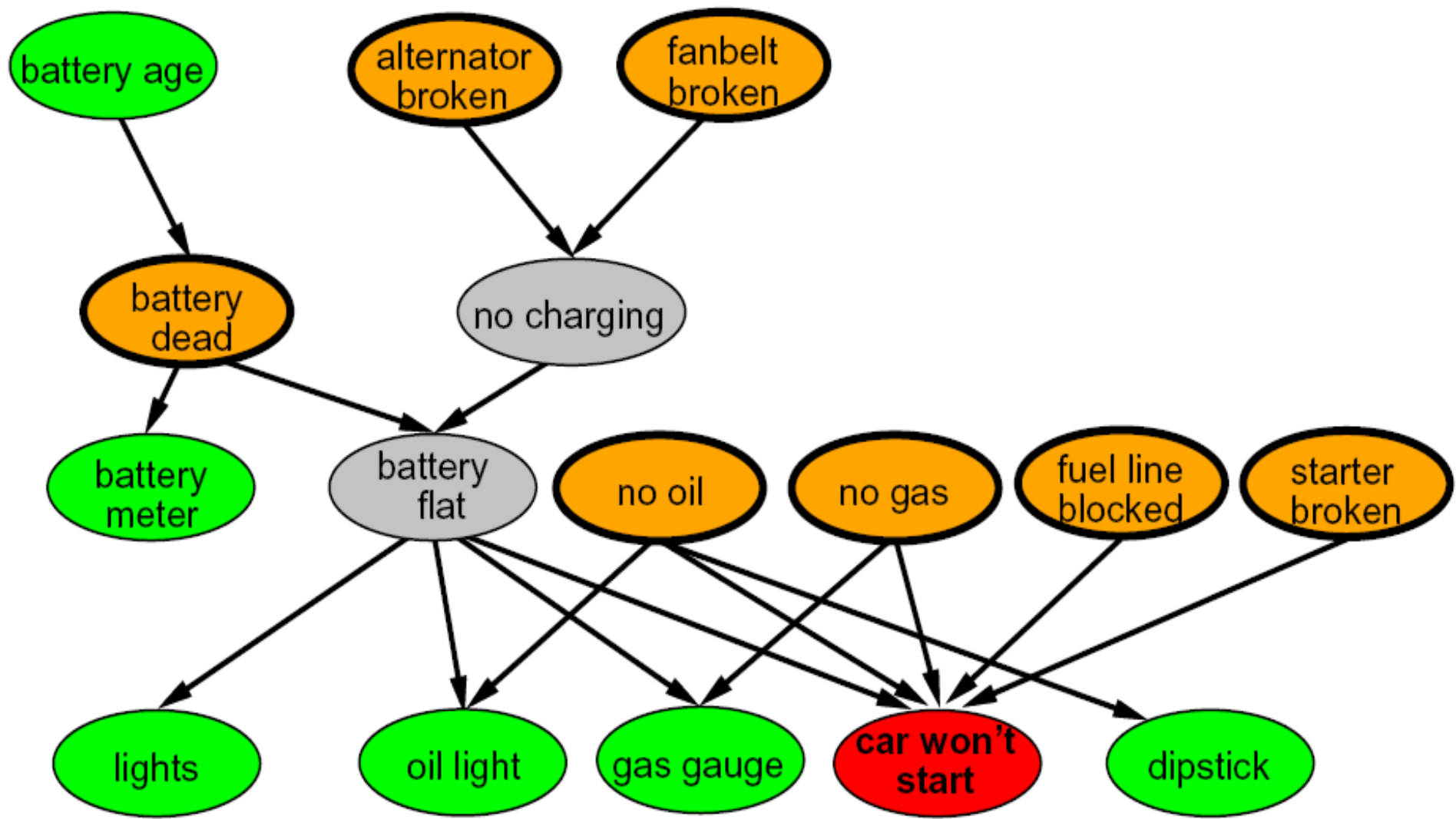


Figure 13.9 A Bayesian network for evaluating car insurance applications.

Example Bayes' Net: Car won't start



Example Bayes' Net:
Liver disorder diagnosis

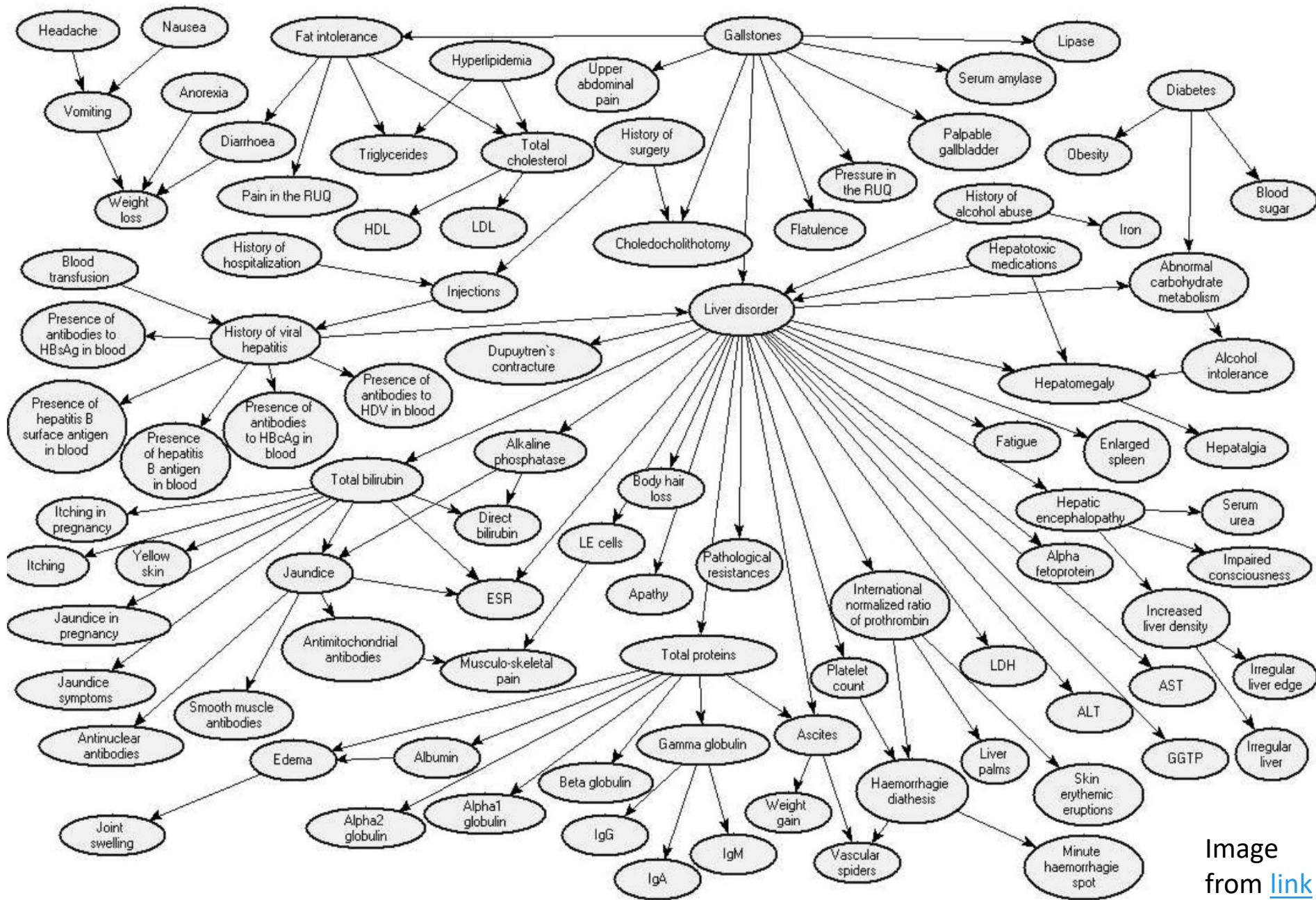


Image from [link](#)

Example Bayes' Net: Printer fault diagnosis

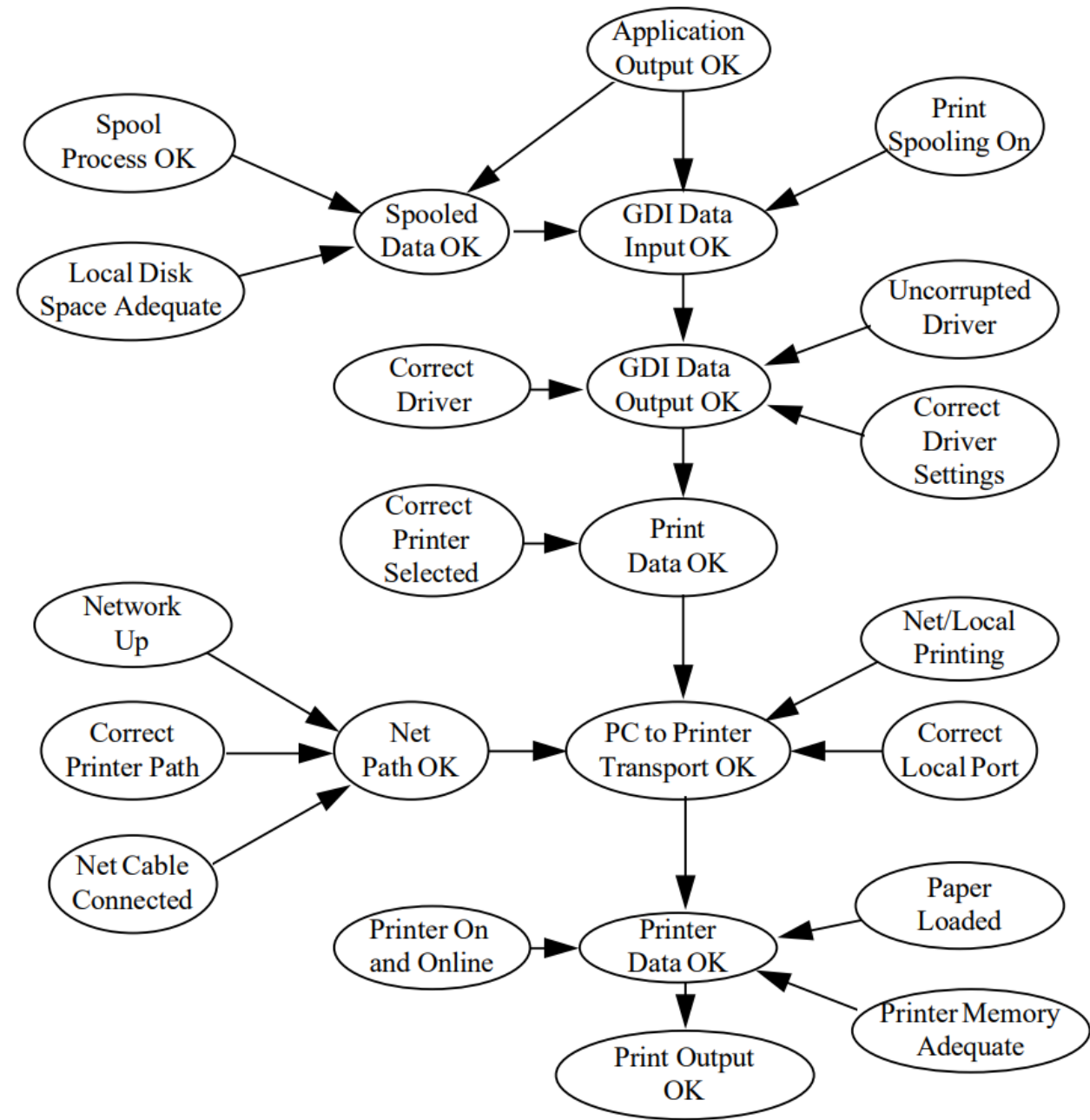


Image from [link](#)
See also [link](#)

1,4	2,4	3,4	4,4
1,3	2,3	3,3	4,3
1,2 B OK	2,2	3,2	4,2
1,1 OK	2,1 B OK	3,1	4,1

- Let's consider a 4x4 grid environment with just breezes and pits.
- Let $P_{i,j}$ denote pit in grid $[i,j]$. For syntax, we'll use $p_{i,j}$ to denote there is a pit and $\neg p_{i,j}$ for no pit in $[i,j]$.
- Let $B_{i,j}$ denote breeze in grid $[i,j]$. For syntax, we'll use $b_{i,j}$ to denote there is a breeze and $\neg b_{i,j}$ for no breeze in $[i,j]$.
- In terms of probability modelling, the full-joint probability for this 4x4 grid environment involves 16 pit random variables + 16 breeze random variables = 32 random variables:

$$Prob(P_{1,1}, P_{1,2}, \dots, P_{4,4}, B_{1,1}, B_{1,2}, \dots, B_{4,4})$$

- We can reduce complexity by making the following reasonable assumptions:
 - $Prob(p_{i,j}) = 0.2$ this is saying that probability that a grid contains a pit is 0.2 in this environment.
 - A breeze in $[i,j]$ is **caused** by some adjacent pit (can be caused by 1 pit, 2 pits, 3 pits, or 4 pits):

$$Prob(b_{i,j} \mid p_{i-1,j} \vee p_{i+1,j} \vee p_{i,j-1} \vee p_{i,j+1}) = 1 \quad \text{Note there are 7 possible ways for a breeze to be caused.}$$

- Special case: there can be no breeze in $[i,j]$ iff there is no pit in adjacent grids (up, down, left, right):

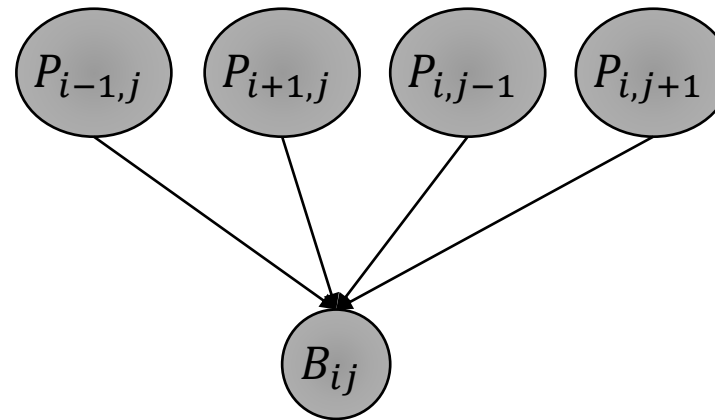
$$Prob(b_{i,j} \mid \neg p_{i-1,j}, \neg p_{i+1,j}, \neg p_{i,j-1}, \neg p_{i,j+1}) = 0$$

- Pits are independent of each other.
- We *cannot* say that breezes are independent of each other, because some breeze grids can share an adjacent pit. We *can* say that breezes are conditionally independent of each other *given* both their adjacent pit statuses.

- Using these assumptions, we get the following:

$$Prob(P_{11}, P_{12}, \dots, P_{44}, B_{11}, B_{12}, \dots, B_{4,4}) = \prod_{ij} Prob(P_{ij}) \prod_{ij} Prob(B_{ij} \mid P_{i-1,j}, P_{i+1,j}, P_{i,j-1}, P_{i,j+1})$$

- What does the BN look like for the 4x4 variant?



1,4	2,4	3,4	4,4
1,3	2,3	3,3	4,3
1,2 B OK	2,2	3,2	4,2
1,1 OK	2,1 B OK	3,1	4,1

- BN will have 16 nodes for the pits, and 16 nodes for the breezes. A breeze node will have at most 4 parent pits (2-3 parents for the boundary breezes). A pit node will have at most 4 breeze children (2-3 children for boundary pits).
- The full-joint has 2^{32} versus the model with these assumptions has $16 + 16 \times 2^4 = 272$ entries. For this particular model, we can get away with just 3 entries:
 - For $Prob(P_{ij})$ we just need 1 entry since it is the same value for all pits 0.2
 - For $Prob(B_{ij} \mid P_{i-1,j}, P_{i+1,j}, P_{i,j-1}, P_{i,j+1})$ we just need 2 entries since it is the same for all breezes (1 in case of pit in some adjacent, 0 in case of no adjacent pit).

Let us look at how to compute $Prob(b_{i,j})$

$$Prob(b_{ij}) = \sum Prob(b_{ij}, P_{11}, \dots, P_{i-1,j}, P_{i+1,j}, P_{i,j-1}, P_{i,j+1}, \dots, P_{44}, B_{11}, \dots, B_{44}) \quad \text{Sum over hidden variables}$$

$$= \sum Prob(b_{ij}, P_{i-1,j}, P_{i+1,j}, P_{i,j-1}, P_{i,j+1} \mid \underbrace{P_{11}, \dots, P_{44}}_{\text{Here we mean the remaining model variables that are not } b_{ij}, P_{i-1,j}, P_{i+1,j}, P_{i,j-1}, P_{i,j+1}}, \underbrace{B_{11}, \dots, B_{44}}_{\text{Notice that these remaining variables do not influence } b_{ij}, P_{i-1,j}, P_{i+1,j}, P_{i,j-1}, P_{i,j+1}}) Prob(P_{11}, \dots, P_{44}, B_{11}, \dots, B_{44}) \quad \text{Product rule}$$

Here we mean the remaining model variables that are not $b_{ij}, P_{i-1,j}, P_{i+1,j}, P_{i,j-1}, P_{i,j+1}$
 Notice that these remaining variables do not influence $b_{ij}, P_{i-1,j}, P_{i+1,j}, P_{i,j-1}, P_{i,j+1}$.

$$= \sum Prob(b_{ij}, P_{i-1,j}, P_{i+1,j}, P_{i,j-1}, P_{i,j+1}) \sum Prob(P_{11}, \dots, P_{44}, B_{11}, \dots, B_{44}) \quad \text{We get independent summation terms. The second term are all hidden variables and summing this distribution will just give 1.}$$

$$= \sum Prob(b_{ij}, P_{i-1,j}, P_{i+1,j}, P_{i,j-1}, P_{i,j+1}) \quad \text{In other words, we just need to sum over hidden variables that influence the query and evidence variables.}$$

$$= \sum Prob(b_{ij} \mid P_{i-1,j}, P_{i+1,j}, P_{i,j-1}, P_{i,j+1}) Prob(P_{i-1,j}) Prob(P_{i+1,j}) Prob(P_{i,j-1}) Prob(P_{i,j+1})$$

Let us look at how to compute $Prob(b_{i,j})$

$$Prob(b_{ij}) = \sum Prob(b_{ij}, P_{i-1,j}, P_{i+1,j}, P_{i,j-1}, P_{i,j+1})$$

In other words, we just need to sum over hidden variables that influence the query and evidence variables.

$$= \sum Prob(b_{ij} \mid P_{i-1,j}, P_{i+1,j}, P_{i,j-1}, P_{i,j+1}) Prob(P_{i-1,j}) Prob(P_{i+1,j}) Prob(P_{i,j-1}) Prob(P_{i,j+1})$$

We have 2^4 settings for hidden variables $P_{i-1,j}, P_{i+1,j}, P_{i,j-1}, P_{i,j+1}$.

$$= Prob(b_{ij} \mid p_{i-1,j}, p_{i+1,j}, p_{i,j-1}, p_{i,j+1}) Prob(p_{i-1,j}) Prob(p_{i+1,j}) Prob(p_{i,j-1}) Prob(p_{i,j+1}) + \dots \\ + Prob(b_{ij} \mid \neg p_{i-1,j}, \neg p_{i+1,j}, \neg p_{i,j-1}, \neg p_{i,j+1}) Prob(\neg p_{i-1,j}) Prob(\neg p_{i+1,j}) Prob(\neg p_{i,j-1}) Prob(\neg p_{i,j+1})$$

$$= (1 \times 0.2^4) + (1 \times 0.2 \times 0.2 \times 0.2 \times 0.8) + (1 \times 0.2 \times 0.2 \times 0.8 \times 0.2) + \dots + (0 \times 0.8^4)$$

The conditional prob. of breeze is 1 when some pit is present, otherwise zero.

$$= 0.5904$$

What about $Prob(b_{ij} \mid \neg p_{i-1,j}, \neg p_{i,j+1})$

$$Prob(b_{ij} \mid \neg p_{i-1,j}, \neg p_{i,j+1}) = \sum Prob(b_{ij}, P_{i+1,j}, P_{i,j-1} \mid \neg p_{i-1,j}, \neg p_{i,j+1})$$

$$= \sum Prob(b_{ij} \mid \neg p_{i-1,j}, \neg p_{i,j+1}, P_{i+1,j}, P_{i,j-1}) Prob(P_{i+1,j}) Prob(P_{i,j-1})$$

We have 2^2 settings for hidden variables $P_{i+1,j}, P_{i,j-1}$.

$$= Prob(b_{ij} \mid \neg p_{i-1,j}, p_{i+1,j}, p_{i,j-1}, \neg p_{i,j+1}) Prob(p_{i+1,j}) Prob(p_{i,j-1}) + \dots \\ + Prob(b_{ij} \mid \neg p_{i-1,j}, \neg p_{i+1,j}, \neg p_{i,j-1}, \neg p_{i,j+1}) Prob(\neg p_{i+1,j}) Prob(\neg p_{i,j-1})$$

$$= (1 \times 0.2^2) + (1 \times 0.2 \times 0.8) + (1 \times 0.8 \times 0.2) + (0 \times 0.8^2)$$

$$= 0.36$$

We just need to sum over hidden variables that influence the query and evidence.

Note that breezes are influenced (caused) by pits, but pits are not influenced by breezes.

What about $Prob(b_{ij} \mid \neg p_{i-1,j}, \neg p_{i+1,j}, \neg p_{i,j+1})$

$$Prob(b_{ij} \mid \neg p_{i-1,j}, \neg p_{i+1,j}, \neg p_{i,j+1}) = \sum Prob(b_{ij} \mid \neg p_{i-1,j}, \neg p_{i+1,j}, \neg p_{i,j+1}, P_{i,j-1}) Prob(P_{i,j-1})$$

We have 2 settings for hidden variable $P_{i,j-1}$.

$$\begin{aligned} &= Prob(b_{ij} \mid \neg p_{i-1,j}, \neg p_{i+1,j}, p_{i,j-1}, \neg p_{i,j+1}) Prob(p_{i,j-1}) \\ &\quad + Prob(b_{ij} \mid \neg p_{i-1,j}, \neg p_{i+1,j}, \neg p_{i,j-1}, \neg p_{i,j+1}) Prob(\neg p_{i,j-1}) \end{aligned}$$

$$= (1 \times 0.2) + (0 \times 0.8)$$

$$= 0.2$$

As we gather more evidence of no adjacent pits, the probability of breeze goes down.

- Consider the situation in the figure. The agent explored grids [1,1], [1,2], and [2,1]. All were safe, breeze detected in [1,2] and [2,1].
- Using propositional logic as we saw in last topic, asking the KB, would just reply that there is a pit in [1,3] or [2,2] or [3,1], which is not helpful.
- Using a probability model, we can ask $Prob(p_{13} \mid evidence)$, $Prob(p_{22} \mid evidence)$, $Prob(p_{31} \mid evidence)$.
- Then move to grid that has lowest pit probability. This allows the agent to manage risk.
- What is the evidence in this current scenario? The info the agent knows about:
 - $\neg b_{11}$ no breeze in [1,1].
 - $\neg p_{11}$ no pit in [1,1].
 - b_{12} breeze in [1,2].
 - $\neg p_{12}$ no pit in [1,2].
 - b_{21} breeze in [2,1].
 - $\neg p_{21}$ no pit in [2,1].

1,4	2,4	3,4	4,4
1,3	2,3	3,3	4,3
1,2 B OK	2,2	3,2	4,2
1,1 OK	2,1 B OK	3,1	4,1

Using a probability model, what is $Prob(p_{13} \mid evidence)$?

$$Prob(p_{13} \mid evidence) = Prob(p_{13} \mid \neg b_{11}, \neg p_{11}, b_{12}, \neg p_{12}, b_{21}, \neg p_{21})$$

$$= \frac{Prob(p_{13}, \neg b_{11}, \neg p_{11}, b_{12}, \neg p_{12}, b_{21}, \neg p_{21})}{Prob(\neg b_{11}, \neg p_{11}, b_{12}, \neg p_{12}, b_{21}, \neg p_{21})}$$

$$= \frac{\sum Prob(p_{13}, \neg p_{11}, \neg p_{12}, \neg p_{21}, P_{22}, P_{31}, \neg b_{11}, b_{12}, b_{21})}{\sum Prob(P_{13}, \neg p_{11}, \neg p_{12}, \neg p_{21}, P_{22}, P_{31}, \neg b_{11}, b_{12}, b_{21})}$$

We just need to sum over hidden variables that influence the query and evidence.
Note that breezes are influenced (caused) by pits, but pits are not influenced by breezes.

$$= \frac{Prob(p_{13})Prob(\neg p_{11})Prob(\neg p_{12})Prob(\neg b_{11} \mid \neg p_{12}, \neg p_{21}) \sum Prob(P_{22})Prob(P_{31})Prob(b_{12} \mid \neg p_{11}, p_{13}, P_{22})Prob(b_{21} \mid \neg p_{11}, P_{22}, P_{31})}{Prob(\neg p_{11})Prob(\neg p_{12}) \sum Prob(P_{13})Prob(P_{22})Prob(P_{31})Prob(b_{12} \mid \neg p_{11}, P_{13}, P_{22})Prob(b_{21} \mid \neg p_{11}, P_{22}, P_{31})}$$

Common terms in numerator and denominator cancel out. Also, $Prob(\neg b_{11} \mid \neg p_{12}, \neg p_{21}) = 1$

$$= \frac{Prob(p_{13}) \sum Prob(P_{22})Prob(P_{31})Prob(b_{12} \mid \neg p_{11}, p_{13}, P_{22})Prob(b_{21} \mid \neg p_{11}, P_{22}, P_{31})}{\sum Prob(P_{13})Prob(P_{22})Prob(P_{31})Prob(b_{12} \mid \neg p_{11}, P_{13}, P_{22})Prob(b_{21} \mid \neg p_{11}, P_{22}, P_{31})}$$

1,4	2,4	3,4	4,4
1,3	2,3	3,3	4,3
1,2 B OK	2,2	3,2	4,2
1,1 OK	2,1 B OK	3,1	4,1

$$= \frac{Prob(p_{13}) \sum Prob(P_{22}) Prob(P_{31}) Prob(b_{12} \mid \neg p_{11}, p_{13}, P_{22}) Prob(b_{21} \mid \neg p_{11}, P_{22}, P_{31})}{\sum Prob(P_{13}) Prob(P_{22}) Prob(P_{31}) Prob(b_{12} \mid \neg p_{11}, P_{13}, P_{22}) Prob(b_{21} \mid \neg p_{11}, P_{22}, P_{31})}$$

$numerator = 0.2 \times ((0.2 \times 0.2) + (0.2 \times 0.8) + (0.8 \times 0.2)) = 0.072$

The denominator are two terms, when $P_{13} = true$ and $P_{13} = false$. We already calculated when $P_{13} = true$, it is the numerator.

$denominator = 0.072 + (0.8 \times ((0.2 \times 0.2) + (0.2 \times 0.8))) = 0.232$

$Prob(p_{13} \mid \neg b_{11}, \neg p_{11}, b_{12}, \neg p_{12}, b_{21}, \neg p_{21}) = \frac{0.072}{0.232} = 0.31$

By similar process we get:

$Prob(p_{31} \mid evidence) = 0.31$

$Prob(p_{22} \mid evidence) = 0.86$

Based on these probabilities, which grid should the agent explore first?

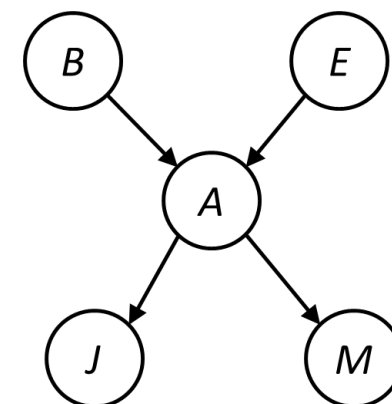
Bayesian Networks – Exact Inference

Exact Inference by **enumeration** in Bayes Net

- Recall that the **full joint distribution** table over all random variables under consideration in a **probability model** gives us all information required to compute any event.
- Given the full joint distribution table $P(X_1, \dots, X_n)$, we can perform inference by enumeration by summing entries from the joint distribution: $P(Q \mid e) = \sum_h P(Q, h, e) / \sum_h P(h, e)$
- A Bayes network compactly represents the full-joint distribution using conditional probabilities. The space complexity is reduced to linear in number of variables $O(n d^k)$.
- What about time-complexity of answering a query in a BN?
- Burglar Network Example:

$$\begin{aligned} P(B \mid j, m) &= \alpha \sum_e \sum_a P(B, e, a, j, m) \\ &= \alpha \sum_e \sum_a P(B) P(e) P(a \mid B, e) P(j \mid a) P(m \mid a) \end{aligned}$$

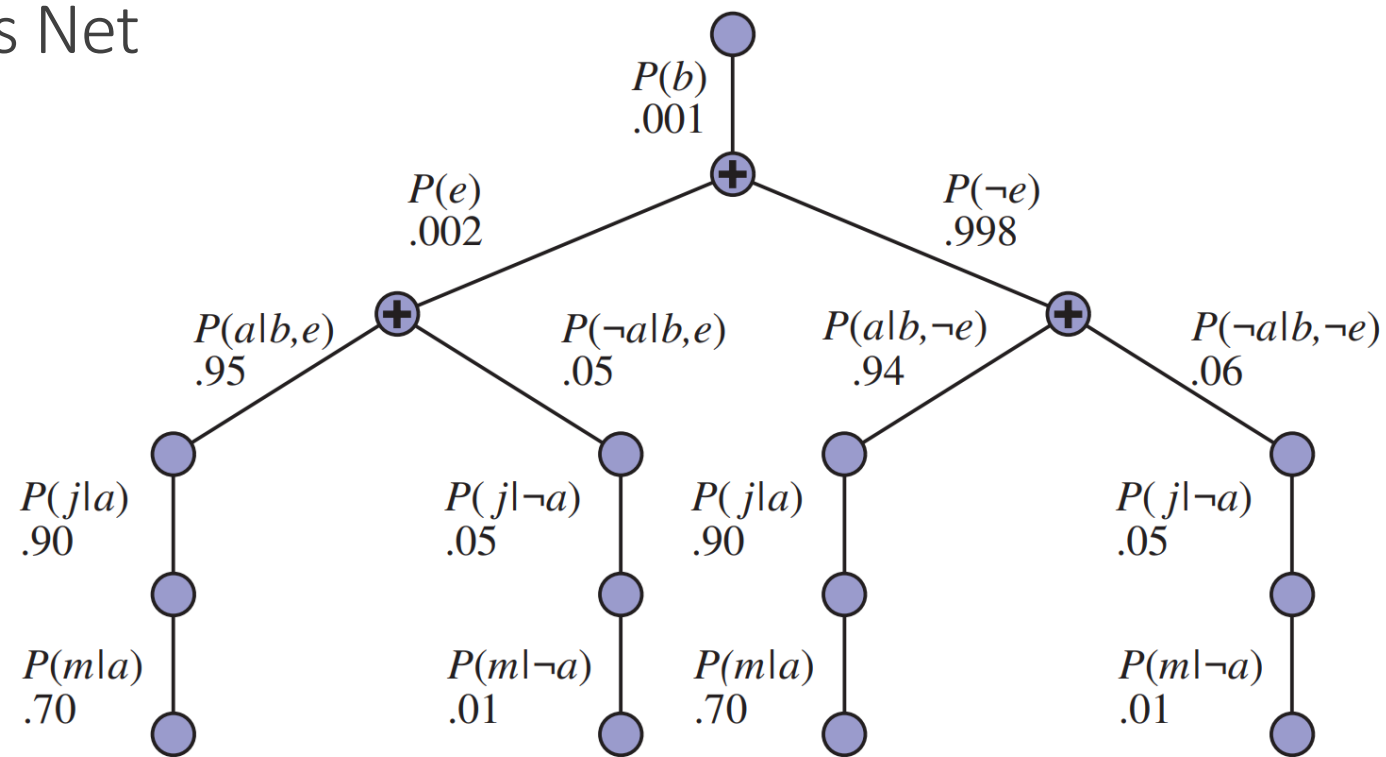
- Issues with this approach:
 - We have to add four terms (2 for e and 2 for a), each with 5 multiplications.
 - In worst-case, we have to sum over $n - 1$ hidden variables; so $O(2^n)$ sums, each a product of $O(n)$ probabilities.
 - So, total worst-case runtime can be $O(n 2^n)$; more work than full-joint due to products.



Exact Inference by **enumeration** in Bayes Net

- So, total worst-case runtime can be $O(n 2^n)$
- We can get rid of the n by re-arranging the terms and moving sums inward.

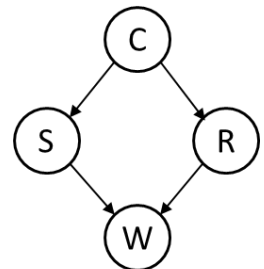
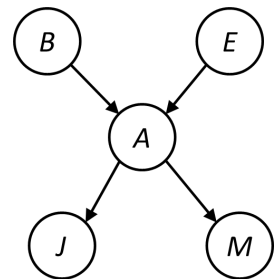
$$\begin{aligned}
 P(B \mid j, m) &= \alpha \sum_e \sum_a P(B, e, a, j, m) \\
 &= \alpha \sum_e \sum_a P(B) P(e) P(a \mid B, e) P(j \mid a) P(m \mid a) \\
 &= \alpha P(B) \sum_e P(e) \sum_a P(a \mid B, e) P(j \mid a) P(m \mid a)
 \end{aligned}$$



- Computation tree shown. This is a DFS approach:
 - Space complexity is linear in number of variables. (DFS)
 - For a general BN and query, worst-case time complexity is $O(d^n)$.
- Can we do better? If you look at the computation tree, you see that the product $P(j|a)P(m|a)$ is computed twice. Similarly, for $P(j|\neg a)P(m|\neg a)$.
- The key to efficient Bayes net inference is to avoid these re-computations. (hint: dynamic programming)

Exact inference in Bayes Nets [1]

- Can we do better? If you look at the computation tree, you see that the product $P(j|a)P(m|a)$ is computed twice. Similarly, for $P(j|\neg a)P(m|\neg a)$.
- The key to efficient Bayes net inference is to avoid these re-computations. (hint: dynamic programming)
- **Variable elimination algorithm** does this
 - Dynamic programming-like approach; avoids re-computations.
 - We won't discuss details of variable elimination (for those interested see [video](#)).
 - Its time-complexity depends on structure of BN and order of variable processing.
 - *Worst-case* can be exponential $O(d^n)$.
 - However, average-case it works well in practice with good ordering using heuristics.
- For the burglar network, we see there is **at most** one path between any two nodes.
 - Such networks are called **singly connected networks** or **polytrees**.
 - Time complexity of exact inference in polytrees is **linear** in the size of the network ($O(n d^k)$; n variables and max k parents per variable). This can be done using variable elimination with a particular ordering that is efficient for polytrees.
- For **multiply connected networks** (which is the general case for BNs):
 - It is NP-hard. No known efficient algorithm exists. *Worst-case* is exponential $O(d^n)$.



Bayesian Networks – Approximate Inference

Approximate Inference in Bayes Net

- Exact inference in BNs is intractable in general. However, we can consider approximate inference.
- Approximate inference in BN uses **random sampling** to compute approximate answers; the more samples we generate, the more accurate the answer.
- Random sampling algorithms are also called Monte-Carlo algorithms.
- Two main families of sampling algorithms:
 - Direct sampling algorithms like prior sampling, rejection sampling, and likelihood weighting.
 - Markov chain sampling algorithms like MCMC and Gibbs sampling. (not covered in this course)
- We will discuss the three direct sampling algorithms mentioned above when trying to approximate probabilities. E.g. Given a BN network, what is the $\text{Prob}(\text{QueryVars} \mid e)$?

Approximate Inference in Bayes Net – Sampling Basics

- To sample a discrete random variable C :
 - Step 1: Take the outcomes of C and their probabilities, and associate each outcome with an interval in the range $[0, 1)$. See how this is done for the example given below.
 - Step 2: Sample u from a uniform distribution over $[0, 1)$. You can use `np.random.uniform()`.
 - Step 3: See where the value of u falls into the intervals defined in step 1. Grab the outcome that u falls into

Example:

C	$P(C)$
red	0.6
green	0.1
blue	0.3

$0.0 \leq u < 0.6, \rightarrow C = \text{red}$

$0.6 \leq u < 0.7, \rightarrow C = \text{green}$

$0.7 \leq u < 1.0, \rightarrow C = \text{blue}$



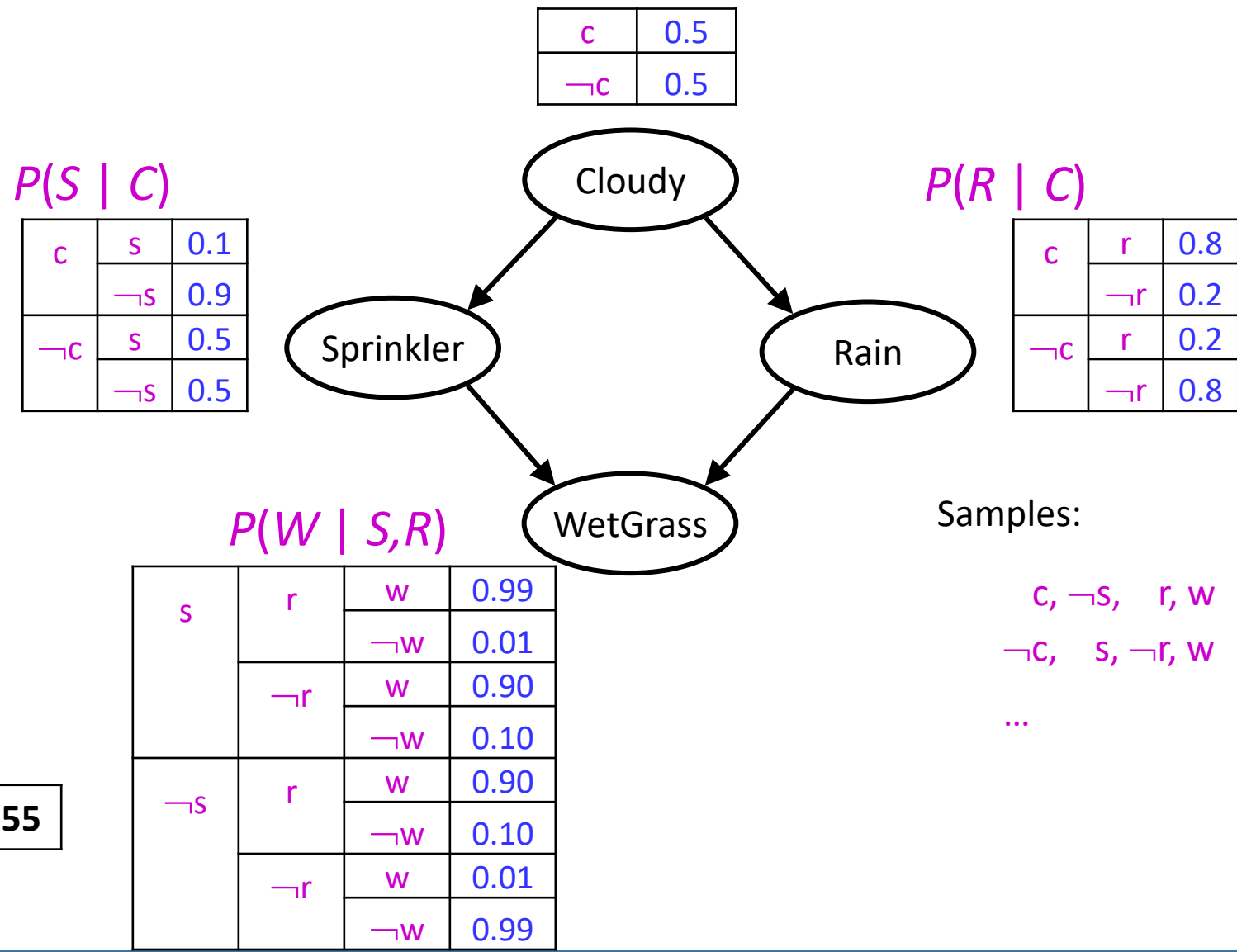
Examples of sampling outcomes for C :

- If `np.random.uniform()` returns $u = 0.83$, then what is sample for C ? $C = \text{blue}$
- If `np.random.uniform()` returns $u = 0.65$, then what is sample for C ? $C = \text{green}$
- If `np.random.uniform()` returns $u = 0.1$, then what is sample for C ? $C = \text{red}$

Approximate Inference in Bayes Net – Prior sampling

- In prior sampling, we sample from a BN based on its structure:
 - Sample variables with no parents first.
 - Then sample variables as their evidence becomes available.
- Consider the BN to the right. To generate a sample:
 - Sample in this order: Cloudy, Sprinkler, Rain, then WetGrass.
 - Another valid sample order: Cloudy, Rain, Sprinkler, then WetGrass.
- Suppose we sample the following values for u , what are the resulting samples from the BN? Use sample order: Cloudy, Sprinkler, Rain, then WetGrass.

0.35	0.7	0.15	0.85	0.9	0.25	0.42	0.55
------	-----	------	------	-----	------	------	------



Samples:

c, $\neg s$, r, w
 $\neg c$, s, $\neg r$, w
...

Approximate Inference in Bayes Net – Prior sampling

For $i = 1, 2, \dots, n$ in topological order

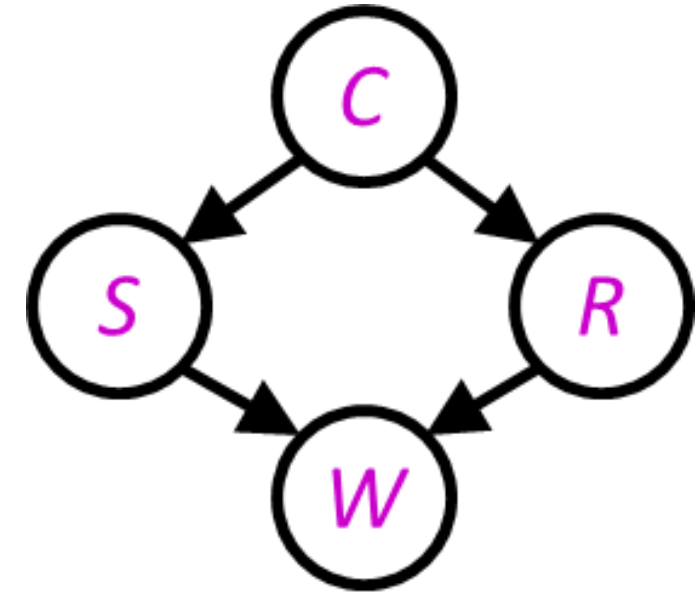
- Sample x_i from $P(X_i \mid \text{Parents}(X_i))$

Return $(x_1, x_2, \dots, x_n)$

Approximate Inference in Bayes Net – Prior sampling Example [1]

- Consider BN to the right.
- Suppose we want to answer $P(\neg s \mid r, w)$?
- We can use prior sampling. Suppose we sample the following:

$C, \neg S, r, w$	←	Among these samples, how many have $\neg s$?
C, s, r, w	←	
$\neg C, s, r, \neg w$		
$C, \neg S, r, w$	←	
$\neg C, \neg S, \neg r, w$		

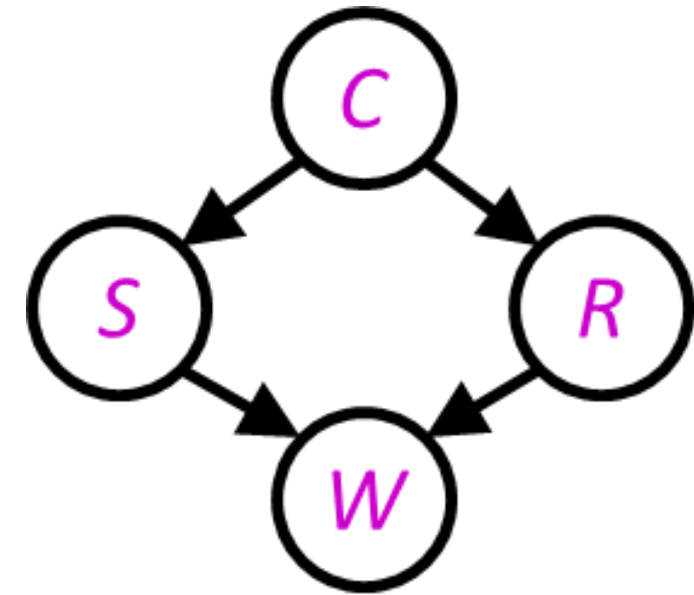


- What is $P(\neg s \mid r, w)$?
 - Grab samples consistent with evidence r, w . Among these samples, how many have $\neg s$?
 - Thus, $P(\neg s \mid r, w) \approx \frac{2}{3}$
 - This will get closer to the *exact* answer with more samples

Approximate Inference in Bayes Net – Prior sampling Example

- Consider BN to the right.
- Suppose we want to answer $P(w)$?
- We can use prior sampling. Suppose we sample the following:

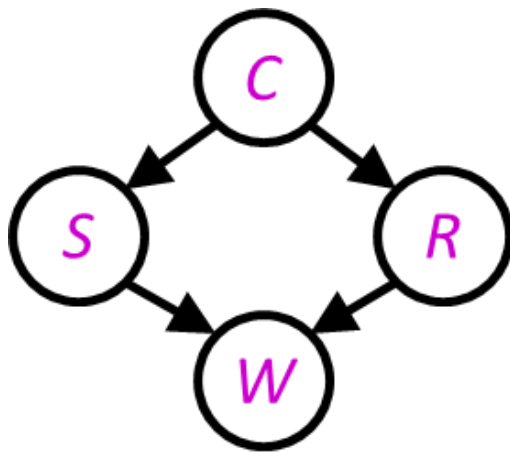
$c, \neg s, r, w$
 c, s, r, w
 $\neg c, s, r, \neg w$
 $c, \neg s, r, w$
 $\neg c, \neg s, \neg r, w$



- What is $P(w)$?
 - With *no evidence*, we consider all the samples. We have counts $\langle w:4, \neg w:1 \rangle$
 - Thus, $P(w) \approx \frac{4}{5}$
 - This will get closer to the *exact* answer with more samples

Approximate Inference in Bayes Net – Rejection Sampling

- **Rejection sampling** is *on-the-fly* application of prior sampling for estimating conditional probabilities
 - Let's say we want to compute $P(c|\neg s, r)$
 - As we generate samples, samples with s or $\neg r$ are not relevant (**reject/discard these samples**)
 - Prior sampling will generate N samples **completely**. Rejection sampling will **discard** inconsistent samples **as it generates them**.
 - So, rejection sampling can be used to generate N samples consistent with the evidence. Making use of all N samples for approximating the conditional query.



$c, \neg s, r, w$
 ~~c, s~~
 $\neg c, \neg s, r, \neg w$
 ~~$c, \neg s, \neg r$~~
 $\neg c, \neg s, r, w$

As we generate a single sample, if we sample a value that is inconsistent with evidence, we can immediately stop and move on to next sample.

Approximate Inference in Bayes Net – Rejection sampling

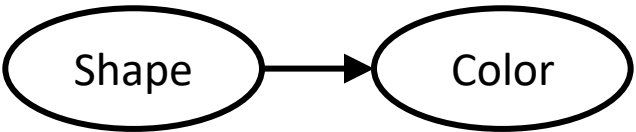
- Input: evidence $e_1, \dots, e_k$
- For $i=1, 2, \dots, n$
 - Sample X_i from $P(X_i \mid \text{parents}(X_i))$
 - If x_i not consistent with evidence
 - Reject: Return, and no sample is generated in this cycle
- Return $(x_1, x_2, \dots, x_n)$

$$P(X \mid e) \approx \frac{N_{PS}(X, e)}{N_{PS}(e)}$$

Approximate Inference in Bayes Net – Likelihood Weighting

Problem with rejection sampling:

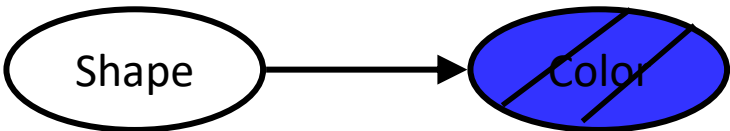
- If evidence is unlikely, rejects **lots** of samples
- $P(\text{evidence})$ drops exponentially as number of evidence variables increases.
- Thus, to get good approximation, it may require large number of samples particularly if evidence is rare.
- For example, suppose that blue is a very rare color, then $P(\text{Shape} \mid \text{Color}=\text{blue})$ would reject almost all samples.



~~pyramid, green~~
~~pyramid, red~~
sphere, blue
~~cube, red~~
~~sphere, green~~

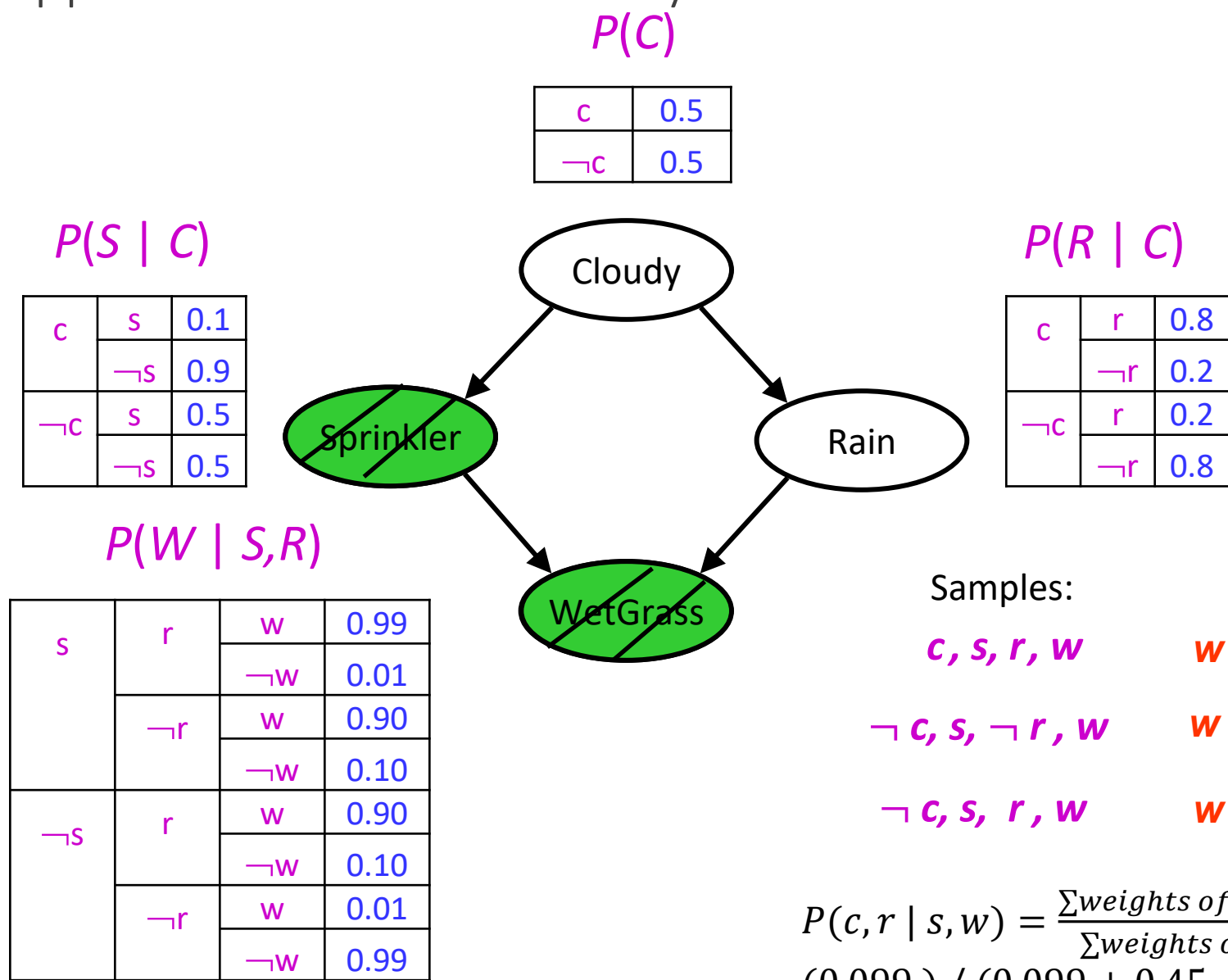
Idea: fix evidence variables, sample remaining variables

- Problem: sample distribution not consistent with true joint distribution! (we are not sampling directly from the BN)
- Solution: apply correction; **weight** each sample by probability of evidence variables given parents (using the BN probabilities)



pyramid, blue	$w=P(\text{blue} \mid \text{pyramid})$
pyramid, blue	$w=P(\text{blue} \mid \text{pyramid})$
sphere, blue	$w=P(\text{blue} \mid \text{sphere})$
cube, blue	$w=P(\text{blue} \mid \text{cube})$
sphere, blue	$w=P(\text{blue} \mid \text{sphere})$

Approximate Inference in Bayes Net – Likelihood Weighting



- What is $P(c, r | s, w)$?
- First fix s and w for each sample. Then sample remaining variables using BN.
- **Weight** each sample by probability of evidence variables given parents.
- To answer the query, use the weights of the samples.

Samples:

c, s, r, w	$w = 1.0 \times 0.1 \times 0.99 = 0.099$
$\neg c, s, \neg r, w$	$w = 1.0 \times 0.5 \times 0.9 = 0.45$
$\neg c, s, r, w$	$w = 1.0 \times 0.5 \times 0.99 = 0.495$

$$P(c, r | s, w) = \frac{\sum \text{weights of samples that have } (c,r,s,w)}{\sum \text{weights of samples that have } (s,w)} = \frac{(0.099)}{(0.099 + 0.45 + 0.495)} = 0.0948$$

Approximate Inference in Bayes Net – Likelihood Weighting

- Input: evidence $e_1, \dots, e_k$
- $w = 1.0$
- for $i=1, 2, \dots, n$
 - if X_i is an evidence variable
 - x_i = observed value for X_i
 - Set $w = w * P(x_i \mid \text{parents}(X_i))$
 - else
 - Sample x_i from $P(X_i \mid \text{parents}(X_i))$
- return $(x_1, x_2, \dots, x_n), w$

Practice – The following samples have been sampled and weighted appropriately. Using likelihood sampling, answer the following queries.

<u>Samples</u>	weights	<u>Queries</u>
+a -b -c -d	0.3	$P(+d) \approx \frac{\sum \text{weights of samples that have } (+d)}{\sum \text{weights of all samples}} = \frac{1.3}{3.8}$
-a +b +c -d	0.4	
+a -b +c -d	0.1	$P(+a, -b) \approx \frac{\sum \text{weights of samples that have } (+a, -b)}{\sum \text{weights of all samples}} = \frac{0.8}{3.8}$
+a +b -c -d	0.3	
+a -b +c +d	0.4	$P(-a, -b, -c, -d) \approx \frac{\sum \text{weights of samples that have } (-a, -b, -c, -d)}{\sum \text{weights of all samples}} = \frac{0.2}{3.8}$
-a -b -c +d	0.1	
-a -b -c -d	0.2	$P(-c \mid -d) \approx \frac{\sum \text{weights of samples that have } (-c, -d)}{\sum \text{weights of samples that have } (-d)} = \frac{1.5}{2.5}$
+a +b +c -d	0.5	
-a +b -c -d	0.7	$P(+d \mid -a, -b) \approx \frac{\sum \text{weights of samples that have } (-a, -b, +d)}{\sum \text{weights of samples that have } (-a, -b)} = \frac{0.1}{0.3}$
+a +b -c +d	0.8	

Notes on Approximate Inference using Sampling

- **Sampling-efficiency**: is the number of samples (or amount of work) an algorithm needs to approximate a value within some error tolerance. $\text{abs}(\text{approxval} - \text{trueval}) \leq \epsilon$
- Sample-efficiency can be used to compare algorithms. We prefer an algorithm that does *less* work to approximate a value.
- When we say algorithm A is *more* sample-efficient than algorithm B, we are saying that A uses less samples than B to achieve same tolerance of approximation value.
- Rejection sampling is *more* sample-efficient than prior sampling.
- Likelihood sampling uses all samples, so it is *more* **sample-efficient** than prior and rejection sampling. (prior and rejection discard some samples during generation)
- However, still degrades as the number of evidence variables increases, but less severe the rejection sampling. By degrades, we mean that to get within the specified tolerance, more samples are needed as the number of evidence variables increases.
- MCMC and Gibbs sampling alleviate degradation issues in likelihood sampling. (not covered in this course)

Bayesian Network python package

- [pgmpy](#) python package for Bayesian networks.
- Supports Approximate (Sampling Based) and Exact inference.

```
>>> from pgmpy.inference import VariableElimination
>>> from pgmpy.models import BayesianNetwork
>>> import numpy as np
>>> import pandas as pd
>>> values = pd.DataFrame(np.random.randint(low=0, high=2, size=(1000, 5)),
...                       columns=['A', 'B', 'C', 'D', 'E'])
>>> model = BayesianNetwork([('A', 'B'), ('C', 'B'), ('C', 'D'), ('B', 'E')])
>>> model.fit(values)
>>> inference = VariableElimination(model)
>>> phi_query = inference.query(['A', 'B'])
```

```
query(variables, evidence=None, virtual_evidence=None,
elimination_order='greedy', joint=True, show_progress=True)
```

```
rejection_sample(evidence=[], size=1, include_latents=False,
seed=None, show_progress=True, partial_samples=None) [so
```

```
likelihood_weighted_sample(evidence=[], size=1,
include_latents=False, seed=None, show_progress=True, n_jobs=-1)
```

Navigation

[Getting Started](#)

[Base Model Structures](#)

[Models](#)

[Parameterization](#)

[Exact Inference](#)

[Model Testing](#)

[Approximate Inference](#)

- [Approximate Inference Using Sampling](#)
- [Bayesian Model Sampling](#)
- [Gibbs Sampling](#)

[Parameter Estimation](#)

[Structure Learning](#)

[Metrics for testing models](#)

[Reading/Writing to File](#)

[Example Notebooks](#)

[Tutorial Notebooks](#)

Summary

Summary [1]

- Representing uncertainty is useful in knowledge bases
- Probability provides a coherent and rigorous formalism for uncertain knowledge
- Joint probability distribution specifies probability of every atomic event
- Any query can be answered using the full-joint distribution, but full joint distributions are intractable to work with.
- For nontrivial domains, must find a way to reduce the joint size:
 - independence and conditional independence provide the tools
 - allow us to model real-world phenomena with much simpler models
- Bayesian networks provide systematic way to construct structured distributions that compress the full-joint distribution.

Summary [2]

- Bayesian networks provide systematic way to construct structured distributions that compress the full-joint distribution.
- However, exact inference in BNs in general is NP-hard.
 - With polytree BNs can use variable elimination in linear time-complexity.
- Approximate inference in BNs can be done using sampling:
 - Direct sampling methods like prior sampling, rejection sampling, and likelihood weighting.
 - Monte-carlo methods like MCMC and Gibbs sampling. (not covered in this course)
- Sampling methods converge to the exact probabilities as the number of samples increases.
- Likelihood sampling is more efficient than rejection sampling since it makes use of all the samples.
- See this python package for probabilistic graphical modelling:
https://pgmpy.org/exact_infer/ve.html